arXiv:2611.06908v1 [cs.RO] 4 Nov 2026

Perceptive Humanoid Motion from Simulation

Siyuan Hu, Abulikemu Abuduweili, Yue Meng, Shipeng Liu, Quentin Leboutet

Chen Chen, Matthias Müller, and Vladlen Koltun

Apple



Fig. 1: **Autonomous whole-body behavior in real-world environments the robot has never seen.** In each environment, the robot is acting in response to a natural-language instruction. The robot sees only its own egocentric camera feed. No scene is scanned and no trajectory is scripted.

***Abstract*— Humanoid robots have the potential to automate a wide range of physical tasks. Progress toward generalist humanoid autonomy, however, remains constrained by a data bottleneck: mapping pixels and free-form language to low-level joint commands requires paired supervision, and no corpus of such data exists at sufficiently large scale. Some approaches rely on human teleoperation and wearable capture devices, which require at least an hour of human time to produce an hour of data. We present HALO, a humanoid autonomy system powered by a pipeline that synthesizes vision-language-motion data across a thousand procedurally generated homes. The corpus scales with compute rather than human labor. HALO acquires a repertoire of physical skills, including sitting on chairs, navigating to named objects, stepping sideways through half-open doors, crouching and crawling beneath tables, and pushing boxes. It orchestrates these skills to solve composite, long-horizon tasks. Without ever seeing a real home during training, HALO transfers zero-shot to the physical world, achieving a 91% success rate across 360 trials in six unmapped real-world environments. See https://apple-aiml-research.github.io/ml-halo/ for code and videos.**

I. Introduction

Humanoid robots can somersault, dance, and run. Yet none of them can be invited into an unfamiliar living room and asked to sit down. They are agile but blind: a backflip is choreographed in advance and executed against geometry the robot never has to perceive. The tasks that would make a humanoid useful are not like this. They take place in rooms nobody has scanned, among furniture nobody has annotated, and the robot has to find the chair before it can sit on it. Perception enters twice, and at two levels of abstraction. It is semantic, deciding what to do: which of the objects around me does the instruction refer to? And it is physical, deciding how to move: the seat has to be met where it actually is, and the robot's weight transferred onto it without losing balance. Neither level is supervised the way a backflip is. A backflip requires only a reference motion clip and a tracker to follow it. Mapping pixels and free-form language

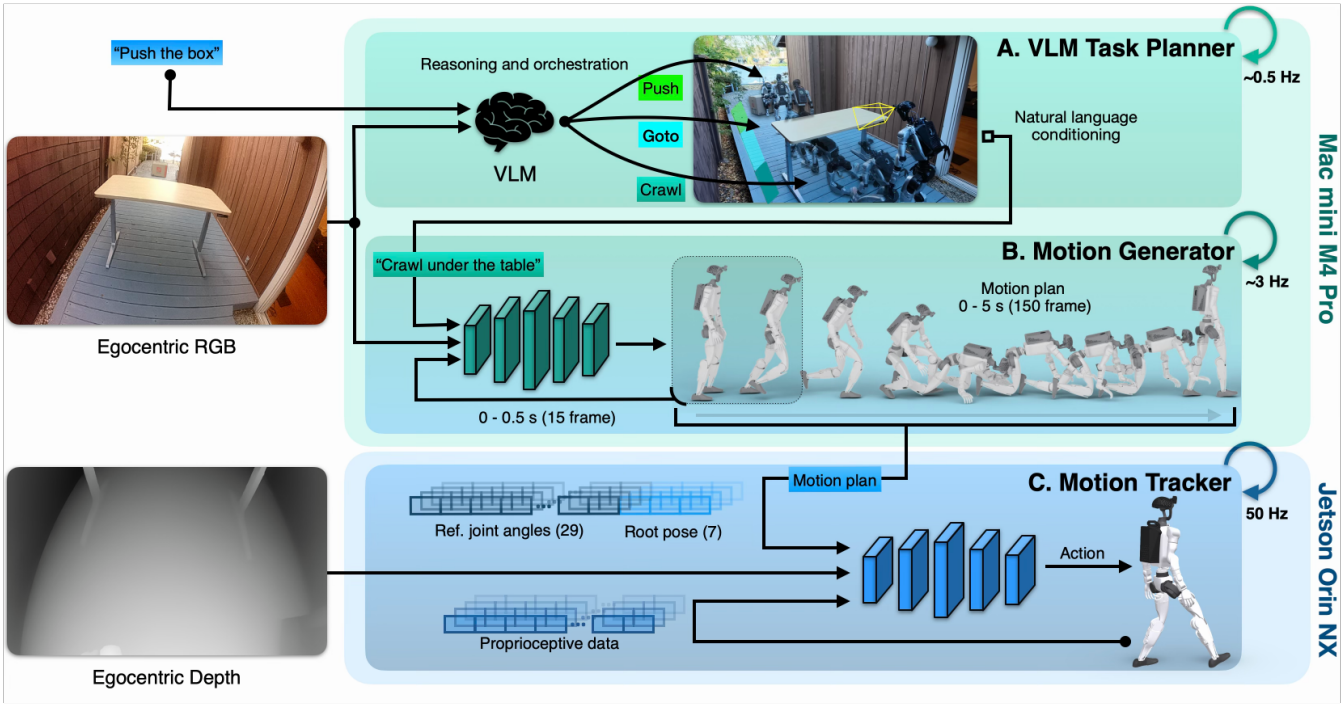


Fig. 2: **Onboard autonomy stack. (A)** A VLM (∼0.5 Hz) processes a natural-language request and egocentric visual input (top left) and outputs a sequence of subtasks. **(B)** The motion generator (∼3 Hz) maps each subtask and the corresponding egocentric RGB frames into a 5 s whole-body motion plan. **(C)** The motion tracker (50 Hz) executes the motion plan, grounding the trajectory into the physical world using egocentric depth (bottom left) and proprioception.

to joint commands, however, demands paired supervision: an egocentric image, an instruction, and the precise whole-body trajectory that satisfies it.

In natural language processing, broad generalization has been unlocked by massive data volume [1]. But while text and video are abundant, as a byproduct of people writing and filming, whole-body motion with paired context is not. Every single episode must be deliberately produced. Teleoperation systems such as ALOHA [2] and TWIST [3], or wearable capture devices such as UMI [4], can generate these demonstrations, but they couple data generation with physical labor. Because a person must manually act out every episode, an hour of data costs at least an hour of human time.

In this paper, we introduce HALO (Humanoid Autonomy, Learned in simulation, running Onboard), a humanoid autonomy system powered by a data generation pipeline that produces egocentric vision-language-motion data entirely in simulation. The resulting policy runs fully onboard the physical robot. Simulation unshackles data generation from human labor. A scene can be generated, a motion synthesized, and the corresponding egocentric video rendered with no human intervention. What would have taken hours (and days, and years) of human time can instead proceed autonomously and in parallel, bounded only by compute. To create a training episode, a motion diffusion model generates the whole-body humanoid motion, a procedural generator supplies the environment, and the simulator situates the motion in the environment to render the robot's sensory inputs. Our training corpus comprises 300 hours of episodes across a thousand procedurally generated homes, none of which anyone has had to visit or scan. Data scales with compute rather than human time. Quality is maintained by automated verifiers.

The HALO autonomy stack has three layers, illustrated in Figure 2. An orchestrator (an off-the-shelf vision-language model invoked every two seconds) decomposes a free-form language request into a sequence of subtasks, each identifying one skill and the object it applies to. A motion generator running at 3 Hz combines a VLM-based encoder with a DiT-based decoder and maps an egocentric RGB observation and the subtask to a whole-body kinematic motion plan: five seconds of joint angles and pelvis poses. Finally, a 50 Hz motion tracker takes this motion plan in conjunction with onboard depth and proprioception, and drives the robot via joint-level actuator commands. The motion tracker is trained by large-scale reinforcement learning to follow motion plans while respecting balance, contact, and actuator limits. The first two layers settle the semantic half of the problem (which chair?); the tracker settles the physical half (meeting the seat where it actually is).

With this stack, HALO autonomously performs perceptive whole-body skills: sitting on chairs, navigating to named objects, stepping sideways through half-open doors, crouching and crawling beneath tables, and pushing boxes. It also orchestrates them into longer multi-step sequences in response to free-form natural-language requests (Figure 1).

This paper makes three contributions:

1) **Data generation pipeline.** HALO produces egocentric

vision-language-motion episodes in simulation, with no teleoperation, no capture suit, and no scan of the deployment environment. Data volume is decoupled from human effort and is bounded only by compute. We train across a thousand procedurally generated home environments.

2) **Onboard autonomy stack.** A high-level orchestrator determines the next semantic subtask. A mid-level generator then solves this task from the current egocentric view, emitting a five-second whole-body motion plan. A low-level tracker grounds this plan in the physical world, executing the trajectory using onboard depth and proprioceptive feedback. No prior system pairs a perceptive low-level controller with a whole-body motion plan produced by a generative model running entirely onboard.
3) **Whole-body autonomy demonstrated in previously unseen physical environments.** Deployed on the physical robot, HALO achieves a 91% success rate over 360 trials across six real-world home environments that were neither seen during training nor scanned or mapped. The system transfers directly from simulation to novel physical environments, with no preparatory scanning of the target environment.

II. Related Work

Motion supervision. Whole-body behavior is typically obtained from recorded human motion, retargeted to the robot's kinematics and tracked on hardware [5]–[7]. Reference trajectories are generally sourced from physical demonstrations, whether captured via optical rigs, teleoperation suits [3], handheld capture devices [4], or raw video [8]. HALO bypasses this manual data collection by generating trajectories directly, using a generative model trained on human motion [9]. A generated trajectory may not be physically realizable. It may slide a foot, penetrate the floor, or float off the ground. Prior methods handle this physical infeasibility at different stages: it can be constrained inside the sampler [10], caught during a physics-based rollout [11], or repaired post-hoc within the dataset [12]. HALO applies geometric verification immediately after sampling, discarding invalid trajectories (Section III-A).

Training environments. Generalizing to unseen homes requires diverse training environments, sourced via reconstruction or procedural generation. If the deployment target is known, reconstruction is highly effective: a brief video scan costs $15\times$ less than teleoperation [13] and often outperforms unrelated diversity [14]. Consequently, systems like VLK train and evaluate exclusively inside pre-scanned rooms [15]. True autonomy, however, demands operating in previously unseen, unmapped spaces. HALO leverages procedural generation to obtain diverse training environments [16], [17]. Prior methods impose strict limits: GRAIL varies terrain but restricts object interactions to authored templates [18], while FetchMan generates $150{,}000$ scenes solely for a single reach-and-pick task [19]. HALO removes these restrictions, training across a thousand procedurally generated homes with different architectural layouts and furniture (Table I).

TABLE I: **Comparison of teleoperation-free humanoid systems.** Blocks group capability (top), training data (middle), and evaluation (bottom). *Training envs.* counts the environments an episode is staged in; ZeroWBC learns from human egocentric video. *Trials* are summed over tasks. $\sim$ marks a capability present in part, or an evaluation on novel objects but not environments; *n/a* marks a quantity that does not apply; – indicates that the quantity was not reported in the paper. HALO is unique in chaining skills online and integrating onboard exteroception directly into the motion tracker.

	ZeroWBC [20]	VLK [15]	GRAIL [18]	TANGO [11]	**HALO (ours)**
Free-form language	✓	×	×	✓	✓
Object interaction	✓	✓	✓	×	✓
Multi-step tasks	×	×	×	×	✓
Perceptive tracker	×	×	$\sim$	×	✓
Training envs.	n/a	2	2	578	1,000
Held-out envs.	$\sim$	×	$\sim$	✓	✓
Real trials / envs.	300 / –	120 / 2	100 / –	45 / 3	**360 / 6**

Hierarchical whole-body control. Systems mapping language to joint commands can be classified in terms of the interface between semantic reasoning and physical control. End-to-end approaches omit this interface entirely, predicting joint targets directly from language or vision [21], [22], or using generated plans solely to shape the training reward [23]. Hierarchical systems use explicit interfaces, such as a compact task-space command [13], [24], sparse body keypoints [25], or a learned latent vocabulary [18], [26]. Architectures closest to HALO pass a full whole-body kinematic trajectory [27]. Prior methods apply these with planning horizons ranging from one second [15] to an entire task [20], and replanning rates ranging from never [20] to 2 Hz [11]. HALO adopts a receding-horizon strategy. It plans five seconds ahead but replans at 3 Hz, executing only the leading part of a trajectory before a fresh observation supersedes it. Furthermore, HALO expands this hierarchy with a high-level orchestrator that translates a free-form natural language request into a sequence of subtasks, advancing upon success and actively retrying upon failure.

Perceptive low-level control. A purely proprioceptive controller is blind to the physical geometry of the environment. Incorporating vision at the low level is a mature line of work [28]–[33], but these perceptive policies are typically trained for narrow, specialized skills like traversing clutter [34]. While some frameworks attempt to generalize by selecting [35] or chaining [36] these skills online, they remain constrained by a finite repertoire. Conversely, general-purpose trackers capable of following an arbitrary reference motion lack perception. Current systems that generate dynamic whole-body references pass them to these blind trackers [7], [8], [11], [15], [20], leaving the robot unable to

react if the physical environment deviates from the generated plan. HALO bridges this gap by introducing a perceptive tracker that integrates egocentric depth to follow generated whole-body reference trajectories.

Sim-to-real evaluation. The true capability of a simulation-trained policy is defined by its evaluation in the physical world: how many distinct real environments it encounters, and whether those environments were seen during training. Prior work reports limited trials in physical environments that were not seen during training (Table I). VLK is evaluated exclusively inside two rooms that were previously scanned and used for training the system [15]. TANGO is tested in three rooms [11], while ZeroWBC and GRAIL report total trial counts without specifying the number of unique environments [18], [20]. HALO establishes a more rigorous protocol. We evaluate six distinct whole-body skills across six real-world environments over 360 physical trials. Crucially, none of these spaces were scanned, reconstructed, or seen during training, ensuring strict zero-shot transfer to hardware.

III. Methods

HALO maps natural language and egocentric vision to low-level joint commands in previously unseen real-world environments. Deployed entirely onboard, it requires no prior scene reconstruction, external motion capture, or privileged object state. We train on diverse procedurally generated synthetic data, described in Section III-A. The onboard autonomy stack has three layers, shown in Figure 2. At the highest layer, a vision-language model (VLM) orchestrates long-horizon behavior by parsing free-form requests into a sequence of linguistic subtasks (Section III-D). Bridging semantics and control, a mid-level motion generator maps the active subtask and egocentric RGB into a whole-body kinematic motion plan (Section III-B). This plan is a 5-second, 30 Hz trajectory specifying 29 joint angles and a relative pelvis pose per frame (36 values total). Finally, a learned motion tracker executes this reference trajectory on hardware using onboard depth and proprioception (Section III-C).

A. Synthetic data generation

The synthetic data generation pipeline is shown in Figure 3 and detailed in Appendix A.

Motion generation. We decompose each task into a sequence of motion segments. Each segment is parameterized by a text prompt, optional waypoint guidance, and a terminal keyframe pose defining the object interaction. For example, the task 'sit on the armchair' breaks down into 'walk forward', 'turn around', and 'sit on chair'. Waypoints for the 'walk forward' segment guide the robot to the front of the seat, while the final segment enforces a seated keyframe anchored to the target asset. This decomposition is written once per task and reused across every procedurally generated environment. See Appendix A for details of the complete segment chain, constraint types, and continuity transformations applied at segment seams. To curate a complete trajectory, we first sample valid locations for the robot and target assets, computing the navigation path via a non-holonomic unicycle planner. We then pass the textual and geometric constraints (waypoints and keyframes) to KIMODO [9], a kinematic motion diffusion model. KIMODO generates dense whole-body motion for each individual segment, which we then stitch into a single, continuous global trajectory. However, because KIMODO operates purely kinematically, its trajectories may not be physically realizable. Left unchecked, the output can exhibit severe artifacts such as foot sliding, floor penetration, or discontinuous jumps. To improve physical plausibility, we generate N candidate trajectories per segment, score them with forward-kinematics geometric verifiers (checking foot skating, floor penetration, flight phases, and joint limits), and retain the highest-scoring candidate.

Scenario staging. To construct the training corpus, we leverage INFINIGEN [17] to procedurally generate 1,000 distinct rooms of three types (living rooms, offices, and bedrooms) with diverse sizes and furniture. Unlike reconstruction-based pipelines [15], this approach requires no physical 3D scans and no manual asset annotation. Environmental diversity scales with compute rather than human labor. To situate a task within a scene, we first extract a 2D occupancy grid from the generated layout. We then jointly embed the synthesized trajectory and its target asset into the scene by sampling $\mathrm{SE}(2)$ transform candidates, accepting the first configuration that avoids collisions between the kinematic trajectory, the asset, and the surrounding furniture. Because every episode is deterministically defined by a random seed and a lightweight placement record, motion generation and rendering are decoupled: the corpus can be re-rendered at new resolutions, with different camera intrinsics, or inside entirely different physics simulators without regenerating the underlying motion.

Egocentric rendering. We kinematically replay staged trajectories in scenes generated by INFINIGEN, rendering egocentric vision through a hardware-matched simulated fisheye lens. To narrow the visual sim-to-real gap, we apply extensive domain randomization encompassing varied color temperature, exposure, contrast, motion blur, and sensor noise (Figure 3C). Because native path-traced rendering is computationally prohibitive at this scale, we bake global illumination and shading into per-object emissive textures. This one-time preprocessing enables interactive rendering via any standard rasterizer, making the pipeline highly scalable and simulator-agnostic. The resulting corpus comprises 300 hours of paired video, language, and motion across 1,000 environments. (For later controlled experiments in simulation, episodes are drawn exclusively from completely held-out environments, rather than novel trajectories within training environments.)

B. Motion generator training

The motion generator is trained on synthetic data (Section III-A). It takes an egocentric RGB image I_t and a language command ℓ as inputs, and produces a full kinematic plan $\hat{q}_{t+1:t+H} \in \mathbb{R}^{H\times 36}$ over a horizon of $H = 150$ frames

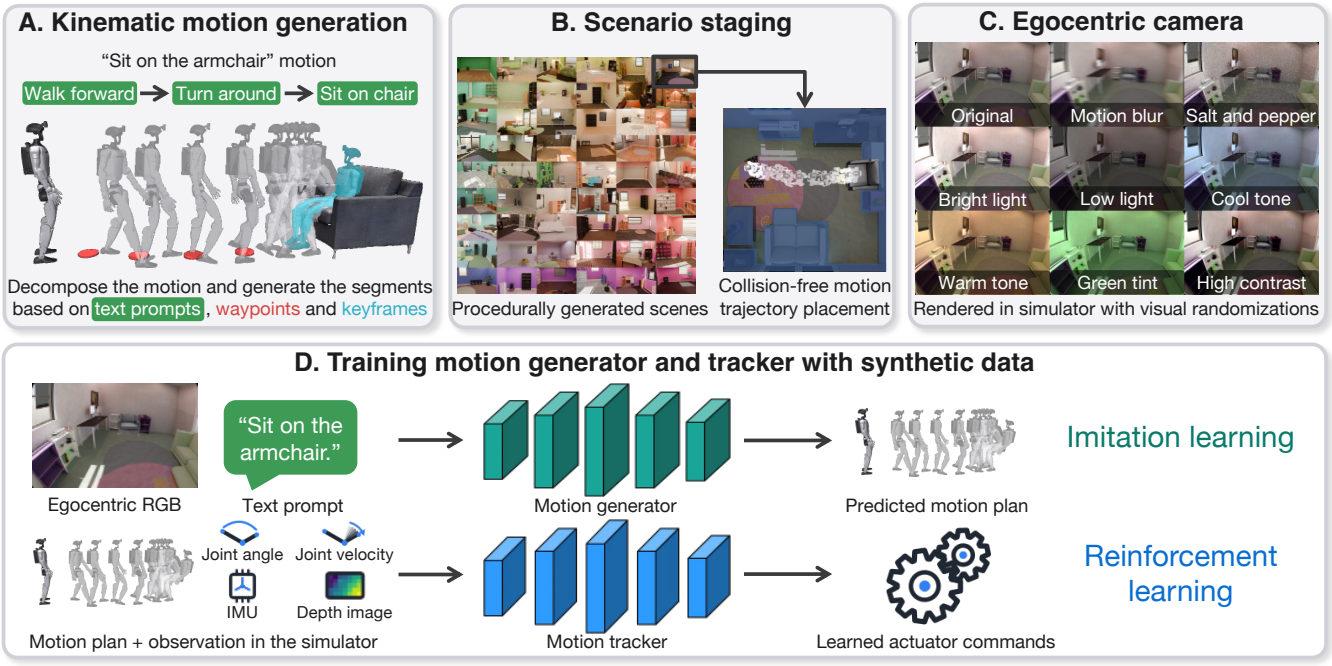


Fig. 3: **Synthetic data generation and training pipeline. (A)** Each task is decomposed into segments defined by text prompts and keyframe poses. After sampling target asset locations and routing navigation waypoints, a conditional diffusion model synthesizes motion segments from these constraints, stitching them into a continuous whole-body trajectory. **(B)** This generated trajectory and its associated assets are embedded into procedurally generated scenes, ensuring collision-free placement. **(C)** A simulator replays the kinematic motion to render egocentric vision from the robot's head-mounted cameras. Visual domain randomization is applied to maximize appearance diversity. **(D)** The resulting corpus is used for training the autonomy stack, via imitation learning for the motion generator and via reinforcement learning for the motion tracker.

(5 seconds at 30 Hz; see Appendix B for architecture and training details). Each frame $\hat{q}_\tau = [\hat{\theta}_\tau; \Delta\hat{p}_\tau; \Delta\hat{r}_\tau]$ stacks the 29 joint angles with a 3D pelvis translation and a 4D pelvis quaternion; both are expressed relative to the pelvis frame at t, so the output is invariant to global placement.

Model architecture. The motion generator is an asymmetric encoder-decoder. The encoder is a Qwen3.5-2B VLM that maps an egocentric RGB image and a natural-language command into a joint feature embedding. Pretrained VLMs already possess rich, web-scale alignment between visual observations and semantic concepts such as 'sit on the chair' or 'push the box'. Keeping the encoder frozen leverages these robust representations while requiring only a single forward pass and no optimizer state for its two billion parameters. The decoder is a diffusion transformer [37] that is trained from scratch (8 blocks, width 512, 8 heads) . It takes as input the VLM feature embedding and generates the dense whole-body motion plan. The decoder is trained via flow matching [38] and sampled using 15 Euler steps during inference.

Asynchronous generation. Closed-loop deployment requires the motion generator to run continuously as new observations arrive. A naive synchronous schedule executes the current plan to completion before invoking the generator on a fresh observation. Because neural inference is not instantaneous, the robot is left without a control signal during the forward pass and must hold its last commanded pose. This stall is inherently unsmooth, fracturing behavior into disjointed segments. It is also physically unsafe: a dynamic system cannot simply pause mid-stride or mid-jump. We therefore run the generator asynchronously. The system computes the next motion plan while the tracker executes the current one. While this eliminates the stall, it introduces a consistency requirement. The current plan spans $[T, T+H)$; the subsequent plan becomes ready at $T+d$, where $0 < d < H$. To prevent kinematic jitter at the swap, this newly generated trajectory must smoothly extend the exact state the robot will reach at $T+d$. Methods like real-time chunking (RTC) [39] attempt to close this seam at inference time by using overlapping frames as an inpainting target. This approach suffers from two critical drawbacks. First, continuity is not mathematically guaranteed. Second, computing this gradient incurs a heavy latency penalty at every denoising step. Instead, we adopt a training-time conditioning formulation [40]. The P=15 frames (0.5 s) the robot is already committed to executing are passed directly into the generator as clean, noise-free tokens. These occupy the first P frames of the sequence, and the model only denoises the remaining $H - P = 135$ frames (4.5 s). Trained to strictly complete a given prefix, the model internalizes continuity natively. Smooth kinematic transitions become an inherent property of the network rather than a costly inference-time patch, allowing fresh motion plans to stream continuously to the low-level tracker.

C. Motion tracker training

We ground kinematic plans into physical execution in three phases: (1) training privileged single-skill experts, (2) distilling them into a unified depth-based student, and (3) fine-tuning via reinforcement learning with actuator modeling and domain randomization for robust sim-to-real transfer (Appendix C).

Phase 1: Privileged experts. We begin by training a dedicated PPO expert for each distinct skill. The experts are trained on our synthesized trajectories (Section III-A) and LAFAN1 human motion clips [41] retargeted to the Unitree G1 [42]. These experts leverage privileged simulation data, observing proprioception, the next 10 planned frames, and ground-truth scene geometry via local height maps. Each expert outputs joint position targets at 50 Hz, which are executed by a low-level PD controller. All experts share an imitation reward that penalizes tracking error on key kinematic links. However, pose imitation alone fails for manipulation. Executing a perfect posture while leaving the target object stationary is a critical failure that purely kinematic rewards cannot detect. Object-centric experts therefore add supplementary terms that reward hand proximity and correct object displacement.

Phase 2: Multi-teacher distillation. We distill the experts into a single student conditioned only on onboard inputs: the motion plan, egocentric depth, proprioception, and the IMU. During distillation, each simulation instance is supervised by the corresponding expert, so the student never averages conflicting actions from different experts. Training follows a hybrid behavior cloning and Dataset Aggregation (DAgger) strategy [43], in which trajectories are collected through a mixture of teacher-driven and student-driven rollouts. A 1D convolution encodes the proprioceptive history, a convolutional network processes the depth image, and a residual MLP maps the resulting latents to target joint positions. The student does not receive object or terrain coordinates and must infer that geometry from depth.

Phase 3: Multi-critic fine-tuning. Distillation is bounded by teacher competence, and partial observability introduces a residual tracking gap. We therefore apply reinforcement learning directly on the student with depth in the loop. To resolve reward-scale disparity across skills, we use multi-critic PPO: the actor is shared, but each skill maintains an independent critic initialized from its Phase 1 expert. Each critic trains exclusively on samples from its skill and normalizes its advantages independently, preventing high-magnitude returns from dominating the policy gradient.

Sim-to-real transfer. Beyond domain randomization, we explicitly model actuator physics to minimize the sim-to-real gap. Following OmniXtreme [44], we analytically derive joint PD gains from motor rotor inertia to guarantee uniform natural frequencies and damping ratios across all joints, and scale actions by joint torque limits. A random action delay simulates hardware command latency. To protect motor drivers during deep crouches, the reward function directly penalizes excessive negative mechanical power at the knees. This keeps the policy from relying on torques that the simulator silently clips but the real motors cannot deliver. Remaining sim-to-real gaps are handled by domain randomization, which is kept narrow in Phases 1 and 2 to preserve teacher fidelity, then widened in Phase 3. We randomize physical parameters including surface friction, restitution, base center-of-mass, actuator gains, joint offsets, external impulse pushes, and object mass. Concurrently, the egocentric depth stream is corrupted with simulated dropout, speckle noise, frame latency, and camera mount misalignments. Unlike blind trackers that trip over unmodeled terrain, our depth-conditioned policy leverages this robust training to actively negotiate geometric variation that the upstream kinematic plan failed to anticipate (Table III).

D. Orchestration for long-horizon tasks

While the generator and tracker excel at atomic, short-horizon behaviors, true autonomy requires satisfying open-ended, long-horizon requests. To achieve this, we top the stack with a VLM-based task orchestrator (Figure 2). Unconstrained natural language presents two fundamental challenges. First, it is inherently underspecified. A command like 'sit down' names no explicit target, forcing the system to infer the intended object directly from the visual context. Second, implicit geometric preconditions often expand a single request into a complex sequence. For instance, 'sit on the chair' translates to a single subtask if the path is clear, but requires a chained sequence of crouching, approaching, and sitting if a table obstructs the robot. To navigate this, the orchestrator processes the egocentric video stream to resolve ambiguous instructions into an ordered plan of actionable subtasks. It drives this sequence in a closed loop. By continuously monitoring the environment, the VLM evaluates the success of the current action, advances the plan when appropriate, and dynamically retries if a failure occurs. At each step, it passes a single, explicit subtask down to the motion generator for execution (Appendix D). By default, the orchestrator runs onboard on the Mac mini using Qwen3.6-35B-A3B (4-bit, MLX), sharing a serving engine with the motion generator (Appendix D), to enable operation without internet connectivity. Optionally, it can route requests to frontier cloud models via their APIs for stronger reasoning and planning capabilities.

IV. Experiments

A. Data and training

The primary training corpus comprises 300 hours of episodes over 1,000 procedurally generated homes (Section III-A). Synthesizing this data requires 48 hours across 250 H100 GPUs, operating without human intervention. The generator trains on 64 B200 GPUs, while the tracker uses 4 H100 GPUs. Held-out episodes (for controlled experiments in simulation) are always staged in completely held-out environments, rather than merely novel trajectories within familiar spaces. Consequently, every performance metric represents generalization to an unseen space.

TABLE II: **Task success in held-out environments.** Single-task evaluation is conducted across 100 simulated and 6 real-world environments, while multi-task evaluation uses 50 simulated and 1 real-world environment. Each task is run for 10 trials per environment. Single-task success requires the robot to complete the task without falling or getting stuck. Multi-task success requires completing all subtasks.

		Success rate (%)	
Category	Task	Sim.	Real
Single-Task	GO-TO	98	92
	SIT	84	85
	DOOR	91	93
	CROUCH	95	100
	CRAWL	84	90
	PUSH	93	88
	Mean	**91**	**91**
Multi-Task	DOOR→GO-TO	71	100
	CROUCH→SIT	73	60
	CROUCH→GO-TO→PUSH	64	50
	Mean	**69**	**70**

B. System

Our hardware platform is a Unitree G1 with two physical modifications: a custom head mount carrying an OAK-1W camera, and a rear enclosure housing a Mac mini M4 Pro (64 GB). Perception and control are distributed across these two machines in a fully untethered, asynchronous cascade linked via ZeroMQ (Appendix E). In the outer loop, the OAK-1W streams a 120° RGB view (1280×800, 30 fps) to the Mac mini, where the orchestrator and motion generator compute the kinematic reference trajectory. This trajectory serves as the setpoint for the inner control loop running on the robot's factory Jetson Orin NX. The tracker consumes the plan alongside a 55° downward-facing Intel RealSense D435i stream (424×240, 30 fps) to issue low-level joint commands. To guarantee strict cycle timing, the tracker is engineered as a C++ real-time process on a `PREEMPT_RT` kernel. It communicates with the hardware interface via lock-free triple buffers, ensuring the control loop never blocks. We bypass the D435i's native noisy depth processing, and route the raw stereo IR pair directly into a TensorRT-accelerated stereo matching model [45] on the Jetson Orin NX. The model estimates clean depth at 30 Hz without the disocclusion or speckle artifacts typical of the native output.

C. Single-task evaluation

These experiments measure whether HALO can execute each task in environments that never appeared in training, as well as on physical hardware. We evaluate six tasks: GO-TO (walk to a named object), SIT (sit on a chair), DOOR (step sideways through a half-open door), CROUCH and CRAWL (pass under a low obstacle), and PUSH (push a box). Each task is evaluated in 100 simulated and 6 real-world environments, with 10 trials per task per environment – 6,000 simulated and 360 real-world trials in total. Table II reports the mean success rate for each task. The policy achieves high single-task transfer (91% sim, 91% real). Some of the real-world environments are shown in Figure 1. Appendix F provides more details on the real-world experiments.

TABLE III: **Stage-wise ablations.** SIT success over 100 held-out simulated environments. 10 trials each. Each block varies one stage; each bold row is HALO's setting (full system), and Δ is the change in success rate relative to it.

Stage	Configuration	Success rate (%)	Δ
Training scenes	**1,000**	**84**	
	100	81	−3
	10	64	−20
	1	43	−41
Real-time chunking	**Training-time**	**84**	
	Inference-time	66	−18
	w/o	24	−60
Generator decoder	**Flow matching**	**84**	
	Regression	19	−65
Motion tracker	**w/ depth**	**84**	
	w/o depth	74	−10

D. Multi-task evaluation

To evaluate long-horizon competence, we run the full system with the orchestrator across three representative multi-task sequences in both simulation and the real world. The results are summarized in Table II. Overall, the VLM's task decomposition is robust for the two- and three-subtask chains we evaluate. Chained success falls below the product of the single-task rates (71% against 89% for DOOR→GO-TO), so the subtasks are not independent.

Finer-grained analysis indicates that the pose in which one subtask ends is an additional factor in multi-task success. Although domain randomization during data generation covers different initial poses, the base position and heading can sometimes be out of distribution. For example, in the real-world CROUCH→GO-TO→PUSH test, 3 of 5 failed trials (60%) end the crouch with the head turned away from the box; the robot never brings it into view and is distracted by other objects. Similarly, two trials of CROUCH→SIT end up trying to sit on the sofa on the right instead of the chair because of the ultimate pose of the crouching segment. The same effect appears in simulation for DOOR→GO-TO: walking through the door succeeds 85% of the time within the chain, but because the door pose varies across scenes, the exit heading varies with it, and the target object can leave the field of view entirely, yielding an 83% conditional success rate for GO-TO and a 71% success rate overall. These cases share a single cause (the next subtask begins with its target unobserved), which suggests adding basic recovery behaviors such as search and turn-in-place to make chaining robust.

E. Ablation study

Table III isolates the contribution of each stage while holding the rest of the system fixed. First, we vary the number of environments in which the motion is staged from 1 to 1,000, holding all other stages fixed and training

policies jointly on all six tasks. This isolates the effect of environment diversity from motion quantity. Scaling the number of environments from 1 to 100 sharply boosts the success rate (43% to 81%), while scaling further yields 3 additional percentage points. Next, without real-time chunking (RTC), the motion plan jumps at every swap and the success rate falls to 24%. Inference-time RTC smooths the transition through a guidance gradient, but only encourages continuity rather than guaranteeing it; it introduces additional inference latency and still trails training-time conditioning by 18 percentage points (66% against 84%). Replacing the flow-matching DiT with a regression decoder using the same transformer backbone lowers the success rate to 19%. RTC does not directly apply to a non-diffusion model, so we lose continuity in the asynchronous generation setting. The regression decoder also overfits easily: outside the training distribution, the plan retains only the coarse root path, losing joint detail, resulting in foot skating. Finally, removing depth leaves only proprioception, lowering the success rate to 74%.

V. Conclusion

We show that simulation can serve as the data engine for humanoid autonomy. Simulation removes the human operator from the data loop, unshackling data generation from human labor. The resulting stack transfers zero-shot to real environments that were neither seen during training nor scanned or mapped.

Limitations and future work. First, our pipeline removes the operator from data collection, but not from task design: adding a new physical task still requires a human to write its prompt and author the keyframes of its motion sequence, so the repertoire grows only as fast as a person can specify additional tasks. Second, the robot's sensory input is largely based on vision, which does not provide direct perception of contact: once an object leaves the camera's field of view or is occluded, the policy has no signal that it is still there, and contact-rich tasks degrade accordingly. Tactile sensing would be a natural solution. Third, the interface between the motion generator and the motion tracker is narrow; the tracker follows the kinematic plan without knowing what the plan is for. Communicating the intended future state of the world would be a natural next step. Finally, our orchestration layer is relatively simple; as the repertoire of tasks grows, we expect to need a more agentic system.

References

[1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," in *NeurIPS*, 2020.

[2] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *RSS*, 2023.

[3] Y. Ze, Z. Chen, J. P. Araújo, Z.-a. Cao, X. B. Peng, J. Wu, and C. K. Liu, "TWIST: Teleoperated whole-body imitation system," in *CoRL*, 2025.

[4] C. Chi, Z. Xu, C. Pan, E. Cousineau, B. Burchfiel, S. Feng, R. Tedrake, and S. Song, "Universal manipulation interface: In-the-wild robot teaching without in-the-wild robots," in *RSS*, 2024.

[5] X. B. Peng, Z. Ma, P. Abbeel, S. Levine, and A. Kanazawa, "AMP: Adversarial motion priors for stylized physics-based character control," *ACM Transactions on Graphics*, vol. 40, no. 4, 2021.

[6] T. He, J. Gao, W. Xiao, Y. Zhang, Z. Wang, J. Wang, Z. Luo, G. He, N. Sobanbab, C. Pan *et al.*, "ASAP: Aligning simulation and real-world physics for learning agile humanoid whole-body skills," in *RSS*, 2025.

[7] L. Yang, X. Huang, Z. Wu, A. Kanazawa, P. Abbeel, C. Sferrazza, C. K. Liu, R. Duan, and G. Shi, "OmniRetarget: Interaction-preserving data generation for humanoid whole-body loco-manipulation and scene interaction," in *ICRA*, 2026.

[8] H. Weng, Y. Li, N. Sobanbabu, Z. Wang, Z. Luo, T. He, D. Ramanan, and G. Shi, "HDMI: Learning interactive humanoid whole-body control from human videos," *arXiv:2509.16757*, 2025.

[9] D. Rempe, M. Petrovich, Y. Yuan, H. Zhang, X. B. Peng, Y. Jiang, T. Wang, U. Iqbal, D. Minor, M. de Ruyter *et al.*, "Kimodo: Scaling controllable human motion generation," *arXiv:2603.15546*, 2026.

[10] Y. Yuan, J. Song, U. Iqbal, A. Vahdat, and J. Kautz, "PhysDiff: Physics-guided human motion diffusion model," in *ICCV*, 2023.

[11] A. Li, Y. Chen, Z. Li, Z. Cao, J. Ren, M. Tomizuka, and D. Shah, "TANGO: Humanoid navigation in cluttered environments with a whole-body vision-language-action model," in *RSS Workshops*, 2026.

[12] M. Xu, Y. Shi, K. Yin, and X. B. Peng, "PARC: Physics-based augmentation with reinforcement learning for character controllers," in *SIGGRAPH*, 2025.

[13] H. Kim, T. Chen, J. Sun, L. W. Osterberg, Q. Chen, K. Wang, and M. Schwager, "LEGS: Fine-tuning teleop-free VLAs for humanoid loco-manipulation in an embodied Gaussian splatting world," *arXiv:2606.01458*, 2026.

[14] M. Deitke, R. Hendrix, A. Farhadi, K. Ehsani, and A. Kembhavi, "Phone2Proc: Bringing robust robots into our chaotic world," in *CVPR*, 2023.

[15] Y.-J. Wang, J. Li, S. Chen, T. E. Truong, P. Xu, P. Abbeel, R. Duan, K. Sreenath, A. Kanazawa, C. Sferrazza *et al.*, "VLK: Learning humanoid loco-manipulation from synthetic interactions in reconstructed scenes," *arXiv:2606.30645*, 2026.

[16] M. Deitke, E. VanderBilt, A. Herrasti, L. Weihs, J. Salvador, K. Ehsani, W. Han, E. Kolve, A. Farhadi, A. Kembhavi *et al.*, "ProcTHOR: Large-scale embodied AI using procedural generation," in *NeurIPS*, 2022.

[17] A. Raistrick, L. Mei, K. Kayan, D. Yan, Y. Zuo, B. Han, H. Wen, M. Parakh, S. Alexandropoulos, L. Lipson *et al.*, "Infinigen Indoors: Photorealistic indoor scenes using procedural generation," in *CVPR*, 2024.

[18] T. Xie, H. Zhang, J. Park, Z. Wang, B. Wen, J. Li, X. Li, Q. Ben, H. Weng, Y. Ye *et al.*, "GRAIL: Generating humanoid loco-manipulation from 3D assets and video priors," *arXiv:2606.05160*, 2026.

[19] O. Rayyan, Z. Li, M. Argus, Y. Jiang, C. Yu, C. Jiang, and Y. Cui, "FetchMan: Learning visual humanoid loco-manipulation policies from simulated experiences," *arXiv:2608.17027*, 2026.

[20] H. Yang, J. Bao, Y. Xin, H. Song, Y. Tian, B. Zhao, D. Wang, and X. Li, "ZeroWBC: Learning natural whole-body humanoid interaction from human egocentric data," *arXiv:2603.09170*, 2026.

[21] X. Xu, Y. Zhang, Y.-L. Li, L. Han, and C. Lu, "HumanVLA: Towards vision-language directed object rearrangement by physical humanoid," in *NeurIPS*, 2024.

[22] T. He, Z. Wang, H. Xue, Q. Ben, Z. Luo, W. Xiao, Y. Yuan, X. Da, F. Castañeda, S. Sastry *et al.*, "VIRAL: Visual sim-to-real at scale for humanoid loco-manipulation," in *CVPR*, 2026.

[23] D. Kalaria, S. S. Harithas, P. Katara, S. Kwak, S. Bhagat, S. Sastry, S. Sridhar, S. Vemprala, A. Kapoor, and J. C.-K. Huang, "DreamControl: Human-inspired whole-body humanoid control for scene interaction via guided diffusion," *arXiv:2509.14353*, 2025.

[24] K. Lin, A. Mandlekar, C. R. Garrett, N. Chernyadev, Y. Fang, R. Ding, Y. Xie, J. Tran, L. Fan, and Y. Zhu, "HumanoidMimicGen: Data generation for loco-manipulation via whole-body planning," *arXiv:2605.27724*, 2026.

[25] S. Yin, Y. Ze, H.-X. Yu, C. K. Liu, and J. Wu, "VisualMimic: Visual humanoid loco-manipulation via motion tracking and generation," *arXiv:2509.20322*, 2025.

[26] H. Xue, X. Huang, D. Niu, Q. Liao, T. Kragerud, J. T. Gravdahl, X. B. Peng, G. Shi, T. Darrell, K. Sreenath *et al.*, "LeVERB: Humanoid whole-body control with latent vision-language instruction," *arXiv:2506.13751*, 2025.

[27] T. He, Z. Luo, X. He, W. Xiao, C. Zhang, W. Zhang, K. Kitani, C. Liu, and G. Shi, "OmniH2O: Universal and dexterous human-to-humanoid whole-body teleoperation and learning," in *CoRL*, 2024.

[28] Z. Zhuang, S. Yao, and H. Zhao, "Humanoid parkour learning," in *CoRL*, 2024.

[29] J. Long, J. Ren, M. Shi, Z. Wang, T. Huang, P. Luo, and J. Pang, "Learning humanoid locomotion with perceptive internal model," in *ICRA*, 2025.

[30] H. Wang, Z. Wang, J. Ren, Q. Ben, T. Huang, W. Zhang, and J. Pang, "BeamDojo: Learning agile humanoid locomotion on sparse footholds," in *RSS*, 2025.

[31] C. Sferrazza, D.-M. Huang, X. Lin, Y. Lee, and P. Abbeel, "Humanoid-Bench: Simulated humanoid benchmark for whole-body locomotion and manipulation," in *RSS*, 2024.

[32] Z. Gu, J. Li, W. Shen, W. Yu, Z. Xie, S. McCrory, X. Cheng, A. Shamsah, R. Griffin, C. K. Liu *et al.*, "Humanoid locomotion and manipulation: Current progress and challenges in control, planning, and learning," *IEEE/ASME Transactions on Mechatronics*, vol. 31, no. 2, 2026.

[33] F. Amadio and E. M. Hoffman, "What matters in humanoid general motion tracking? An empirical study," *arXiv:2607.19903*, 2026.

[34] H. Xue, S. Liang, Z. Zhang, Z. Zeng, Y. Liu, Y. Lian, J. Wang, Q. Liu, X. Shi, and L. Yi, "Collision-free humanoid traversal in cluttered indoor scenes," *IEEE Robotics and Automation Letters*, vol. 11, no. 8, 2026.

[35] H. Chen, Z. Li, H. Wang, J. Hu, Z. Li, P. Liu, Q. Zhao, X. Liu, L. Pan, X. Lyu *et al.*, "Light-Loco-Parkour: Versatile perceptive whole-body locomotion via multi-skill distillation," *arXiv:2608.02653*, 2026.

[36] Y. Lin, J. Cui, Y. Li, B. Jia, Y. Zhu, and S. Huang, "LessMimic: Long-horizon humanoid interaction with unified distance field representations," *arXiv:2602.21723*, 2026.

[37] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *ICCV*, 2023.

[38] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *ICLR*, 2023.

[39] K. Black, M. Y. Galliker, and S. Levine, "Real-time execution of action chunking flow policies," in *NeurIPS*, 2025.

[40] K. Black, A. Z. Ren, M. Equi, and S. Levine, "Training-time action conditioning for efficient real-time chunking," *arXiv:2512.05964*, 2025.

[41] F. G. Harvey, M. Yurick, D. Nowrouzezahrai, and C. Pal, "Robust motion in-betweening," *ACM Transactions on Graphics*, vol. 39, no. 4, 2020.

[42] J. P. Araújo, Y. Ze, P. Xu, J. Wu, and C. K. Liu, "Retargeting matters: General motion retargeting for humanoid motion tracking," *arXiv:2510.02252*, 2025.

[43] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *AISTATS*, 2011.

[44] Y. Wang, S. Zhu, P. Zhi, Y. Li, J. Li, Y.-L. Li, Y. Xiao, X. Wang, B. Jia, and S. Huang, "OmniXtreme: Breaking the generality barrier in high-dynamic humanoid control," *arXiv:2602.23843*, 2026.

[45] J. Jing, W. Luo, Y. Mao, and K. Mikolajczyk, "Lite Any Stereo: Efficient zero-shot stereo matching," in *CVPR*, 2026.

[46] S. Gao, M. Pagnucco, T. Bednarz, and Y. Song, "NVIDIA Isaac Sim: Enabling scalable, GPU-accelerated simulation for robotics," *arXiv:2606.03551*, 2026.

[47] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *IROS*, 2012.

[48] Z. Shen, H. Pi, Y. Xia, Z. Cen, S. Peng, Z. Hu, H. Bao, R. Hu, and X. Zhou, "World-grounded human motion recovery via gravity-view coordinates," in *SIGGRAPH Asia*, 2024.

[49] H. Lee, D. Hwang, D. Kim, H. Kim, J. Tai, K. Subramanian, P. R. Wurman, J. Choo, P. Stone, and T. Seno, "SimBa: Simplicity bias for scaling up parameters in deep reinforcement learning," in *ICLR*, 2025.

[50] NVIDIA, M. Mittal, P. Roth, J. Tigue, A. Richard, O. Zhang, P. Du, A. Serrano-Muñoz, X. Yao, R. Zurbrügg *et al.*, "Isaac Lab: A GPU-accelerated simulation framework for multi-modal robot learning," *arXiv:2511.04831*, 2025.

[51] Z. Wu, X. Huang, L. Yang, Y. Zhang, K. Sreenath, X. Chen, P. Abbeel, R. Duan, A. Kanazawa, C. Sferrazza *et al.*, "Perceptive humanoid parkour: Chaining dynamic human skills via motion matching," *arXiv:2602.15827*, 2026.

[52] Y. Zhang, Y. Seo, J. Chen, Y. Yuan, K. Sreenath, P. Abbeel, C. Sferrazza, K. Liu, R. Duan, and G. Shi, "RPL: Learning robust humanoid perceptive locomotion on challenging terrains," *arXiv:2602.03002*, 2026.

[53] Q. Liao, T. E. Truong, X. Huang, Y. Gao, G. Tevet, K. Sreenath, and C. K. Liu, "BeyondMimic: From motion tracking to versatile humanoid control via guided diffusion," *arXiv:2508.08241*, 2025.

[54] C. Tessler, Y. Guo, O. Nabati, G. Chechik, and X. B. Peng, "MaskedMimic: Unified physics-based character control through masked motion inpainting," *ACM Transactions on Graphics*, vol. 43, no. 6, 2024.

[55] S. Zhao, Y. Ze, Y. Wang, C. K. Liu, P. Abbeel, G. Shi, and R. Duan, "ResMimic: From general motion tracking to humanoid whole-body loco-manipulation via residual learning," *arXiv:2510.05070*, 2025.

[56] Tencent Hunyuan3D Team, "Hunyuan3D 2.1: From images to high-fidelity 3D assets with production-ready PBR material," *arXiv:2506.15442*, 2025.

APPENDIX

A. Data generation

The training corpus in Section III-A consists of six datasets (one per skill), with 162,698 episodes and 296.3 h of data at 30 Hz in total (Table IV). All the skills share the same logic for procedural scene generation (INFINIGEN), motion generation (KIMODO) and egocentric rendering, whereas each skill has its own sequence of KIMODO calls, quality filters and appearance randomization. Each episode has one fixed instruction, which is shared by all the episodes of a skill, except for GO-TO, which has one instruction per target object. This gives ten instructions in the corpus: "sit arm_chair", "walk to armchair", "walk to basketball", "walk to box", "walk to man", "walk to trashbin", "Go through the half-open door", "crawl under the table", "crouch under the table" and "push box". At run time, the orchestrator converts a free-form request into one of these instructions (Appendix D).

Scene generation. We use the indoor generator of INFINIGEN [17] to generate single rectangular rooms, namely living rooms (45%), offices (30%) and bedrooms (25%), with a length of 3.5 to 9.5 m per side. Each room has sparse, medium or dense furniture (20 to 80 objects), one of four floor materials, up to two rugs, two or three windows, and plain or decorated walls (Figure 4–*top*). We generate 1,200 rooms, of which 1,178 finish (the other 22 stop inside the scene generation process, when its furniture solver hangs or crashes). We use 1,000 rooms for training and hold out the remaining 178 rooms, and the simulated evaluation uses 100 of the held-out rooms. DOOR needs a doorway, so it uses 966 two-room and multi-room scenes from the same generator.

Scene baking. Path tracing all 32 million training frames in Blender Cycles is too slow (13.2 s per frame, compared to 56–87 ms in Isaac Sim [46] with real-time ray tracing), so we bake the lighting of each room only once. We first use Cycles to bake the direct and indirect lighting of every object into a texture atlas with 128 samples per pixel. The atlas resolution grows with the object surface area (about 1,024 pixels per meter, up to 4,096 pixels). Then we project eight path-traced views onto the atlases to correct the texels they cover, and finally we replace every material with an unlit emissive one. Since a baked texture cannot represent view-dependent reflections, we make all the wooden surfaces matte, which halves the bake error on a test room. The bake takes 38.5 minutes per room (median) on a single GPU. We measure the bake error as the mean absolute difference between a baked render and the path-traced render of the same view, in grey levels of an 8-bit image (0 to 255). For reference, two path-traced renders of the same view with different random samples already differ by 1.05 grey levels. Over all rooms, the median error is 2.0 on the eight projected views and 3.8 on 16 held-out views that the bake never used, i.e., on a new view a pixel is off by 1.5% on average (Figure 4–*bottom*). This matters because every training image is rendered from the bake, so the bake error adds to the visual gap between simulation and the real camera. The bake also carries over to other renderers: MuJoCo [47] and Isaac Sim render it within 1.5 and 2.0 grey levels of Blender, so we can render the corpus in a fast simulator without baking again. The remaining errors mostly come from glass, mirrors and screens, whose appearance depends on the view. We leave the ceiling out of the bake, since a closed room comes out too dark. The head camera is pitched down by 25°, so the ceiling only appears near the top edge of the images.

Kinematic motion generation. We use KIMODO [9] to generate the motions, with a classifier-free guidance weight of 2.0 on both the text and the constraints (root paths and full-body keyframes in Figure 5). For SIT, we generate the whole episode with the chair at the origin before choosing a room. The robot starts 1.5 to 3 m away from the seat (within ±70° in front of the chair), and a unicycle model plans a path to a point 0.55 m in front of the seat. We then call KIMODO three times: a walk guided by 20 waypoints, a turn with 50 frames, and a sit with 50 frames that ends at a seated pose from a joint-state log of the real G1. Each segment starts from the last frame of the previous segment, and we smooth each transition with a linear cross-fade over three frames on each side. GO-TO is a single walk to one of five objects (trash bin, box, basketball, armchair and a standing person). The walk starts 2 to 4 m away from the object, with the object in view. DOOR is a single call where the robot walks to a door opened by 30° to 45°, side-steps through the gap with three step-together stances, and walks out. CRAWL and CROUCH are also single calls, guided by six keyframes from phone videos of a person going under a table. We lift the videos to 3D with GVHMR [48], retarget them to the G1 with GMR [42], and edit the keyframes by hand. CRAWL goes on hands and knees under tables 0.70 to 1.05 m high, whereas CROUCH goes in a deep stoop under tables 1.05 to 1.40 m high. PUSH connects a generated walk to a motion-capture recording of a person pushing a $0.46 \times 0.50 \times 0.56$ m box, with a bridge of 9 to 11 frames in between.

Filtering. Since KIMODO is purely kinematic, we score the candidates on their forward kinematics. For SIT, we use 33 checks (e.g., foot skate, hops, flight phases, pauses, torso lean, joint limits and the final seated height), and a candidate passes if it fails none of the 23 hard checks and has a weighted score of at least 0.75. We first sample 16 walks in a batch and keep the best one, then generate the turn and the sit. If the turn or the sit still fails after two retries, we resample the walk (up to six rounds) and keep the best chain in the end, even if it still fails. Of the 50,000 generated SIT episodes, 77% pass all the hard checks, and the 25,349 episodes in the corpus are evenly spaced over all the 50,000 episodes. CRAWL and CROUCH use 21 checks with a pass score of 0.65, keep the best of 20 candidates, and drop the episode if no candidate passes. For PUSH, we sample 10 walks for each start and keep the one with the gait closest to the recorded walk.

Scene staging. We place each motion into a room with a rigid 2D rotation and translation, which is applied to both the robot path and the target object. We search up to 800

TABLE IV: **Training corpus by skill.** Every KIMODO call is guided by a root path. KF denotes full-body keyframe constraints.

Skill	Motion	Keyframes from	Filter	Appearance	Episodes	Hours	Mean length
SIT	walk (best of 16), turn, sit	G1 joint log	33 checks, keep best	image randomization	25,349	50.0	7.1 s
GO-TO	one walk to an object	standing pose	target in view at start	random dome light	50,000	47.1	3.4 s
DOOR	one call, 5 KF	hand-edited poses	none	image randomization	29,207	50.0	6.2 s
CRAWL	one call, 6 KF	phone video, GVHMR, GMR	21 checks, best of 20	random dome light	19,066	50.0	9.4 s
CROUCH	one call, 6 KF	phone video, GVHMR, GMR	21 checks, best of 20	random dome light	19,369	49.2	9.1 s
PUSH	walk (best of 10) + recorded push	motion capture	gait ranking	image randomization	19,707	50.0	9.1 s
Total					162,698	296.3	6.6 s

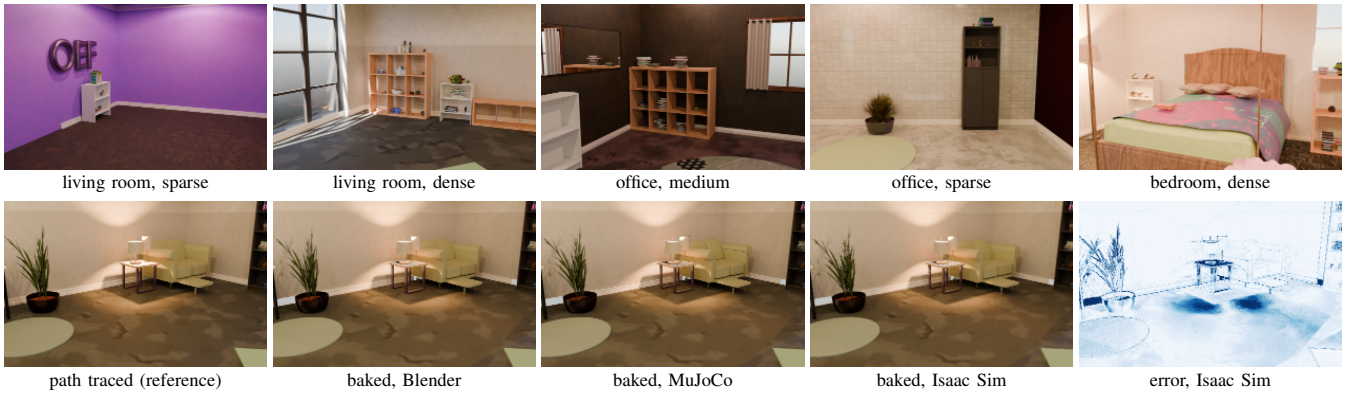


Fig. 4: **Procedural rooms and scene baking.** Top: path-traced training rooms from INFINIGEN. Bottom: a held-out view of a dense living room rendered from the bake in three engines, where the mean absolute error relative to path tracing is 4.2 (Blender), 4.5 (MuJoCo) and 4.7 (Isaac Sim) grey levels and mostly lies in the light pool of the lamp on the floor and at object edges (error map: white is 0, dark blue is 32 grey levels or more).

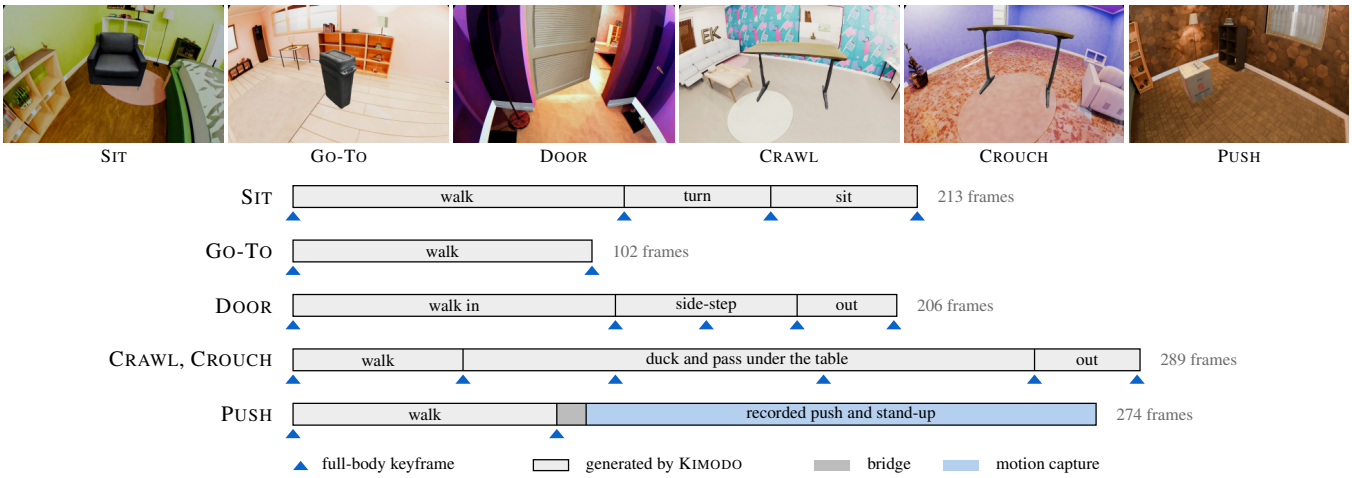


Fig. 5: **Episodes of the six skills.** Top: training frames from the egocentric fisheye camera (1280×800). Bottom: a typical episode of each skill on the same time axis (blue triangles are full-body keyframes), where only SIT chains several KIMODO calls.

free cells on a 10 cm occupancy grid with 24 headings, and accept the first transform where the pelvis (checked every three frames) is at least 0.35 m away from all the obstacles and the target object fits in the room. Since each episode is fully specified by its seed and placement record, we can re-render the corpus with another camera or renderer without regenerating the motion. For DOOR, we instead align the motion to the doorway and remove the furniture on the robot path.

Egocentric view rendering. We replay the episodes kinematically and render them in Isaac Sim 5.1 with real-time ray tracing (PUSH is rendered in MuJoCo). The egocentric camera matches the calibrated OAK-1W fisheye camera on the robot, with 1280×800 pixels and a $128° \times 81°$ field of view, pitched down by 25°. Since the baked lighting is fixed, we randomize the appearance either after rendering or through the dome light. SIT, DOOR and PUSH randomize the exposure, contrast, tint, motion blur (along the camera motion) and sensor noise of the rendered images. GO-TO, CRAWL and CROUCH randomize the intensity and color of

the dome light at render time, which changes the brightness and tint of the room because Isaac Sim also lights the baked textures. For the 50,000 SIT episodes, generating and rendering the episodes and caching their vision-language features takes about 9 h on 250 H100 GPUs, and generating the 1,178 rooms takes 26 h on up to 300 GPUs.

B. Motion generator

Network architecture. The generator follows the encoder-decoder design of Section III-B: a frozen Qwen3.5-2B reads the image and the language command, and a DiT decoder [37] turns its features into a 150-frame motion plan. Table V provides the full configuration.

We pass the 224×224 image and the language command together in a single prompt to Qwen and use the last-layer hidden states of all 64 image tokens and all text tokens.

Image and text tokens each go through their own small projection (linear, LayerNorm, GELU) from Qwen's width of 2048 down to the decoder width of 512. We concatenate them and add a learned positional embedding. The result is the conditioning sequence the decoder attends to.

The decoder generates the plan by flow matching (Appendix B.2): it starts from random noise and refines it into a clean plan over several steps. Its input at each step is therefore the current noisy plan, which has the same shape as the output, 150 frames of 36 values. Each frame becomes one token: a linear layer maps its 36 values to 512, and a sinusoidal embedding encodes the frame index. Each of the 8 blocks has self-attention across the frame tokens, cross-attention to the conditioning sequence, and a feed-forward layer. The flow time s, which says how noisy the plan still is, enters every block through adaptive LayerNorm, which scales and shifts the features of each token. A final linear layer maps each token back to 36 values: the velocity that moves the noisy plan toward the clean one. This layer is initialized to zero, so training starts from a model that predicts zero velocity everywhere. Only the projections and the decoder are trained, 49.3M parameters in total.

Training loss. We train the decoder with flow matching [38]. Joint angles and pelvis translation are first normalized to zero mean and unit variance per dimension; the quaternion is left unchanged. Given a normalized plan x_1 and Gaussian noise x_0, we draw a flow time $s \sim \mathrm{Beta}(1.5, 1)$ and mix the two:

$$x_s = s\, x_1 + (1 - s)\, x_0. \tag{1}$$

At $s = 0$ this is pure noise and at $s = 1$ it is the clean plan. The decoder v_ϕ, with weights ϕ, learns to predict the velocity $x_1 - x_0$ that carries one to the other.

For asynchronous generation, the first $P = 15$ frames are frames the robot is already committed to, so we never noise them. The decoder therefore receives two modified inputs. The first is the plan $\tilde{x}_s$, whose first P frames come from the clean plan x_1 and whose remaining frames come from the noisy plan x_s. The second is the per-frame flow time $\tilde{s}$, which is 1 on the first P frames, marking them as clean, and s on the remaining frames. The loss covers only the frames the decoder has to generate:

$$\mathcal{L}(\phi) = \mathbb{E}\left[\frac{1}{D\,(H-P)} \sum_{\tau=P+1}^{H} \left\| v_\phi(\tilde{x}_s, \tilde{s}, c)_\tau - (x_1 - x_0)_\tau \right\|^2\right], \tag{2}$$

where c is the conditioning sequence, $H = 150$ is the plan length, $D = 36$ is the number of values per frame, and the subscript τ selects frame τ. The model thus learns to continue whatever prefix it is given, and consecutive plans join without a jump. We train for 50 epochs on 31.0M samples using 64 B200 GPUs.

At inference, the first P frames of the new plan, the prefix, are copied from the next P frames of the plan the robot is currently executing. The other $H - P$ frames of the new plan start from noise. We take 15 Euler steps from $s = 0$ to $s = 1$. After each step, we overwrite the first P frames with the prefix again, so the new plan begins exactly where the current one continues.

TABLE V: **Motion generator configuration.**

Group	Parameter	Value
Encoder	backbone	Qwen3.5-2B (frozen)
	image resolution	224×224
	feature layer	last
	conditioning tokens	64 image + all text (≤ 96)
Decoder	architecture	DiT [37]
	blocks / width / heads	8 / 512 / 8
	FFN width	2048
	dropout	0.1
	conditioning	cross-attention
	trainable params	49.3M
Plan	horizon H	150 frames (5 s)
	prefix P	15 frames (0.5 s)
	frame dimension	36 ($29 + 3 + 4$)
	sampler	Euler, 15 steps
Training	optimizer	AdamW, $\beta = (0.9, 0.999)$
	learning rate	10^{-3}, cosine to 10^{-5}
	weight decay	0.01
	gradient clipping	1.0
	batch size	5,120 (80×64 GPUs)
	epochs	50
	flow-time distribution	$\mathrm{Beta}(1.5, 1)$
	precision	FP32

C. Motion tracker

The motion tracker (Section III-C) is a 50 Hz policy that executes the kinematic motion plan from the generator on the robot (Figure 6). On hardware, it runs in TensorRT on the robot's onboard Jetson Orin NX inside a real-time C++ process. At each step, it reads the upcoming frames of the motion plan, joint positions and velocities, IMU readings, and a depth image from the downward-facing RealSense D435i, and outputs 29 target joint positions for the low-level PD controller (simulated at 200 Hz across four physics substeps per policy step, and tracked on hardware by the onboard motor controllers).

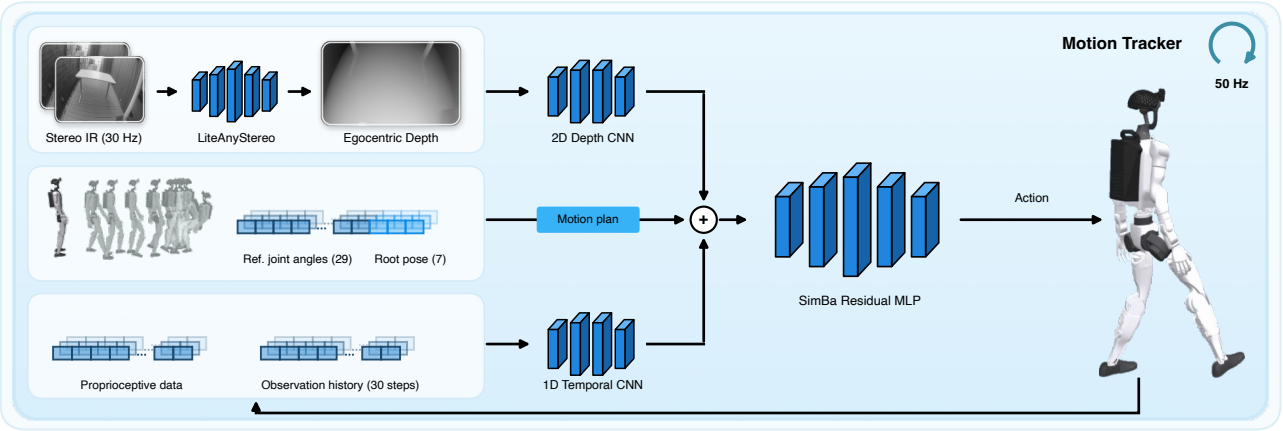


Fig. 6: **Motion tracker onboard inference architecture.** On the Jetson Orin NX, the tracker runs at 50 Hz and fuses three onboard streams: (top left) raw stereo IR frames from the RealSense D435i (emitter off) are mapped by LiteAnyStereo [45] (TensorRT FP16) to a clean 48×84 egocentric depth image and encoded by a 2D CNN; (bottom left) a 30-step proprioceptive history together with the next 30 plan frames is encoded by a 1D temporal CNN; and (center) both feature vectors are concatenated with the current proprioceptive state and 5-frame motion plan lookahead and passed through a SimBa residual MLP [49] to output 29 target joint positions for the low-level PD controller.

Three-phase training. Training a single depth-conditioned policy from scratch across all skills fails because contact dynamics and reward scales vary widely across tasks, and rendering depth slows early RL exploration. We therefore train the tracker in three phases in Isaac Lab [50] (Table VI).

Phase 1: Privileged experts. To avoid negative transfer across disparate behaviors, we first train a separate PPO expert for each skill family: locomotion and obstacle traversal on flat ground (GO-TO, DOOR, CROUCH, CRAWL, and retargeted human motion clips), object manipulation (PUSH), and object interaction (SIT). Each expert observes privileged simulator state, a 10-frame reference window with extremity errors, a 30-step history, and a local height map of the terrain and objects. Episodes start from random frames within each clip; to focus training on hard transitions, starting frames are sampled in proportion to per-segment failure rates [27].

Phase 2: Multi-teacher distillation. We distill the frozen experts into a single student policy that uses only onboard sensors (proprioception, IMU, the motion plan, and egocentric depth) in a shared scene containing flat ground, furniture, and movable objects. At each step t, simulation instance i is routed to the expert $\pi^{\text{teacher}}_{k(i)}$ for its active skill $k(i)$, avoiding the blending artifacts of action averaging. Rollouts follow a hybrid behavior cloning and DAgger strategy [22], [43] (20% teacher-driven rollouts and 80% student-driven rollouts), and the student with weights θ minimizes the squared error between its mean action $\boldsymbol{\mu}_\theta(\mathbf{o}^{\text{stu}}_{i,t})$ and the routed expert's mean action $\boldsymbol{\mu}^{\text{teacher}}_{k(i)}(\mathbf{o}^{\text{priv}}_{i,t})$:

$$\mathcal{L}_{\text{distill}}(\theta) = \mathbb{E}_{i,t}\left[\left\|\boldsymbol{\mu}_\theta\left(\mathbf{o}^{\text{stu}}_{i,t}\right) - \boldsymbol{\mu}^{\text{teacher}}_{k(i)}\left(\mathbf{o}^{\text{priv}}_{i,t}\right)\right\|^2\right]. \quad (3)$$

Phase 3: Multi-critic fine-tuning. Distillation is bounded by teacher competence, and partial observability leaves a residual tracking gap [44], [51]. In Phase 3, we initialize the actor from the distilled student and train it with PPO with depth rendering in the loop across 4 H100 GPUs (16,384 environments). Because object interaction skills include extra object rewards (Table VII), their returns have a different scale than locomotion, which would cause a single shared critic to skew the policy gradient. We resolve this with multi-critic PPO: the student actor is shared across all skills, while K independent privileged critics $\{V_{\psi_k}\}_{k=1}^{K}$, one per skill family, are initialized from the Phase 1 experts' critics. Each critic is updated only on transitions from its own skill, and the generalized advantage estimate $A_{i,t}$ is standardized separately within each skill's batch:

$$\hat{A}_{i,t} = \frac{A_{i,t} - \mu_{k(i)}}{\sigma_{k(i)} + \varepsilon}, \quad (4)$$

where μ_k and σ_k are the mean and standard deviation of the advantages for skill k. The shared actor then updates across all skills using the normalized advantages $\hat{A}_{i,t}$.

TABLE VI: **Motion tracker architecture and training hyperparameters.**

Parameter	Phase 1	Phase 2	Phase 3
Network architecture			
History encoder (1D CNN)	[64, 128, 128]	[32, 64, 128]	[32, 64, 128]
Spatial input	17×11 height	48×84 depth	48×84 depth
Spatial encoder (2D CNN)	[32, 64, 128]	[16, 32, 64, 128]	[16, 32, 64, 128]
SimBa MLP [49] ($L\times W$)	3×768	2×512	2×512
Reference lookahead	10 (0.2 s)	5 (0.1 s)	5 (0.1 s)
Training			
Algorithm	PPO	DAgger (20% teacher)	Multi-critic PPO
Parallel envs (GPUs)	8,192 (1)	4,096 (1)	16,384 (4)
Rollout steps per env	24	32	24
Training iterations	50,000	30,000	50,000
Learning rate (Adam)	10^{-3}	10^{-4}	3×10^{-4}
Epochs / mini-batches	5 / 8	5 / 8	5 / 8
PPO clip / entropy	0.2 / 0.008	—	0.2 / 0.001
Discount γ / GAE λ	0.99 / 0.95	—	0.99 / 0.95

Observations and network architecture. We track $|\mathcal{B}|=14$

body links: the pelvis (which carries the IMU and serves as the anchor link), the torso, and the left and right hip, knee, ankle, shoulder, elbow, and wrist links, with the two ankles and two wrists forming the extremity set $\mathcal{B}_{\text{ee}}$.

Asymmetric observations. The Phase 1 experts and Phase 3 critics observe noiseless simulator state: base linear and angular velocity, projected gravity, joint positions and velocities, the previous action, the 3D poses of all 14 tracked links in the pelvis frame, the pelvis pose error, a 10-frame (0.2 s) future reference window, a 30-step (0.6 s) history, and a $1.6\,\text{m} \times 1.0\,\text{m}$ height map at 0.1 m resolution (17×11). Following RPL [52], heights are measured relative to the feet so the elevation profile of furniture or objects stays constant when the pelvis lowers during crouching or sitting. The deployable student (Figure 6) drops all world-frame velocities and link poses, observing only noisy IMU angular velocity and gravity, joint positions and velocities, the previous action, the pelvis orientation error, a shorter 5-frame (0.1 s) reference window, the next 30 plan frames, a 30-step history, and a 48×84 depth image.

Simulated and onboard depth. Standard depth ray-casters in simulation ignore the robot's own body, which would let a downward-looking head camera see through the arms and legs. Following RPL [52], we render simulated depth (25 Hz) with a two-pass Warp ray-caster that queries both the scene meshes and the robot's articulated link meshes, taking the closer hit at each pixel so the student observes realistic self-occlusion during manipulation, crawling, and sitting. Depth values are clipped to $[0.2, 2.5]$ m and mapped via $\log(1 + d)$. On hardware, the RealSense D435i's built-in stereo matcher produces heavy speckle noise, occlusion shadows, and invalid holes. Rather than simulating these artifacts, we disable the camera's active IR dot projector and stream the raw left and right infrared images at 30 Hz into LiteAnyStereo [45] (7.6 M parameters, compiled to TensorRT FP16 on the Jetson Orin NX). The predicted disparity is converted to metric depth, downsampled to 48×84 by averaging only valid measured pixels within each cell (mapping missing returns to the 2.5 m far plane so holes do not bias nearby obstacles), clipped to $[0.2, 2.5]$ m, and log-transformed before being passed to the 50 Hz control loop through lock-free triple buffers (Figure 6).

Policy network. As shown in Figure 6 and Table VI, a 3-layer 1D CNN encodes the 30-step observation history, and a 4-layer stride-2 2D CNN encodes the 48×84 depth image (or a 3-layer 2D CNN over the 17×11 height map for the experts and critics). Both feature vectors are concatenated with the current state and 5-frame motion plan and passed through a SimBa residual MLP [49] with pre-LayerNorm feed-forward blocks (GELU) and a linear output head to predict the 29 target joint positions.

Rewards and terminations. Phase 1 and Phase 3 optimize the reward in Table VII.

Motion and object rewards. Six Gaussian kernels track the global pelvis pose and the relative poses and velocities of the 14 tracked links $\mathcal{B}$, with relative link targets aligned to the robot's current horizontal pelvis position and yaw so horizontal drift is not penalized twice [53]. For object manipulation (such as pushing a box), pose imitation alone fails when the robot copies the arm motion in free air while leaving the object stationary. Following ResMimic [55] and MaskedMimic [54], we measure the distance from the wrists to the object, $d_{\text{hand}} = \min_{h\in\text{wrists}} \|\mathbf{p}_h - \mathbf{p}_{\text{obj}}\|$, and gate the object tracking terms by $g_{\text{hand}} = \exp(-(d_{\text{hand}}/0.3)^2)$ so the policy is rewarded for moving the object only when the hands are near the object.

TABLE VII: **Reward terms optimized in Phase 1 and Phase 3.** Pose and velocity errors are $\mathbf{e}_{p,b} = \mathbf{p}_b^{\text{ref}} - \mathbf{p}_b$, $\mathbf{e}_{R,b} = \text{Log}(\mathbf{R}_b^{\text{ref}}\mathbf{R}_b^\top)$, $\mathbf{e}_{v,b} = \mathbf{v}_b^{\text{ref}} - \mathbf{v}_b$, and $\mathbf{e}_{\omega,b} = \boldsymbol{\omega}_b^{\text{ref}} - \boldsymbol{\omega}_b$ ($\tilde{\mathbf{e}}$ denotes pelvis-aligned relative errors [53]). Object terms are gated by hand proximity $g_{\text{hand}} = \exp(-(d_{\text{hand}}/0.3)^2)$ [54], [55].

Reward Term	Expression	Weight
Motion tracking [53]		
Pelvis position	$\exp\big(-\|\mathbf{e}_{p,\text{pelvis}}\|^2/0.3^2\big)$	1.0
Pelvis orientation	$\exp\big(-\|\mathbf{e}_{R,\text{pelvis}}\|^2/0.4^2\big)$	1.0
Link positions	$\exp\big(-\frac{1}{\lvert\mathcal{B}\rvert}\sum_{b\in\mathcal{B}}\|\tilde{\mathbf{e}}_{p,b}\|^2/0.3^2\big)$	1.0
Link orientations	$\exp\big(-\frac{1}{\lvert\mathcal{B}\rvert}\sum_{b\in\mathcal{B}}\|\tilde{\mathbf{e}}_{R,b}\|^2/0.4^2\big)$	1.0
Link linear velocities	$\exp\big(-\frac{1}{\lvert\mathcal{B}\rvert}\sum_{b\in\mathcal{B}}\|\mathbf{e}_{v,b}\|^2/0.5^2\big)$	1.0
Link angular velocities	$\exp\big(-\frac{1}{\lvert\mathcal{B}\rvert}\sum_{b\in\mathcal{B}}\|\mathbf{e}_{\omega,b}\|^2/1.5^2\big)$	1.0
Object interaction [54], [55]		
Object position	$g_{\text{hand}}\exp\big(-\|\mathbf{e}_{p,\text{obj}}\|^2/0.3^2\big)$	1.0
Object orientation	$g_{\text{hand}}\exp\big(-\|\mathbf{e}_{R,\text{obj}}\|^2/0.4^2\big)$	1.0
Object drop	$g_{\text{hand}}\max(0,\, z_{\text{obj}}^{\text{ref}} - z_{\text{obj}} - 0.15)$	-1.0
Regularization and hardware safety [44]		
Action rate	$\|\mathbf{a}_t - \mathbf{a}_{t-1}\|_2^2$	-0.05
Joint position limits	$\sum_j \max(0,\, q_j - q_j^{\max},\, q_j^{\min} - q_j)$	-100
Undesired contacts	$\sum_{b\notin\mathcal{B}_{\text{ee}}}\mathbb{I}(\|\mathbf{f}_b^{\text{contact}}\| > 1\,\text{N})$	-0.1
Knee braking power	$\sum_{j\in\text{knees}}\max(0,\, -\tau_j\dot{q}_j - 150)^2/500^2$	-10.0
Stance foot slip	$\sum_{f\in\text{feet}}\|\mathbf{v}_f^{xy}\|\,\mathbb{I}_{\text{stance}}(f)$	-0.2

Safety penalties and terminations. Alongside penalties on action rate, soft joint-limit violations, non-extremity contacts above 1 N, and foot slip during stance, we penalize excessive negative mechanical power $P_j = \tau_j\dot{q}_j < 0$ (regenerative braking) at the knees during deep crouches and sitting. Following OmniXtreme [44], braking power above a 150 W deadband is penalized to prevent policies from exploiting simulator torque clipping and tripping hardware overcurrent protection. Phase 1 terminates failed episodes using strict 3D cutoffs (0.5 m pelvis, 0.3 m extremities $\mathcal{B}_{\text{ee}}$); in Phases 2 and 3, because the student has no global position sensor and naturally drifts horizontally, we relax the 3D pelvis cutoff to 0.8 m and check vertical drift (0.3 m pelvis, 0.375 m extremities) so valid obstacle traversals are not cut short [44], [51].

Actuator modeling and domain randomization. To minimize the sim-to-real gap, we combine system-identified actuator modeling with a two-tier domain randomization schedule (Table VIII).

System identification and PD control. Following BeyondMimic [53] and OmniXtreme [44], we derive nominal joint stiffness $k_{p,j}$, damping $k_{d,j}$, and torque-normalized action scale α_j from each motor's rotor inertia I_j using a natural

TABLE VIII: **Domain randomization ranges across training phases.** Noise and depth rows apply to the student (Phase 2); Phase 1 experts observe noiseless state without depth.

Parameter	Phases 1–2	Phase 3
Actuators and robot inertia [44], [53]		
PD stiffness / damping scale	1.0 / 1.0	$[0.85, 1.15]$ / $[0.75, 1.3]$
Motor armature scale	1.0	$[0.90, 1.20]$
Joint torque limit scale	1.0	$[0.80, 1.05]$
Default joint offset (rad)	± 0.01	± 0.01
Command latency (ms)	$[5, 25]$	$[15, 40]$
Torso CoM Δx / $\Delta y, z$ (m)	± 0.025 / ± 0.05	± 0.025 / ± 0.05
Limb mass scale	1.0	$[0.90, 1.10]$
External velocity pushes (m/s)	± 0.50	± 0.50
Ground surface and objects		
Ground friction (static / dynamic)	$[0.3, 1.6]$ / $[0.3, 1.2]$	$[0.3, 1.6]$ / $[0.3, 1.2]$
Ground restitution	$[0.0, 0.4]$	$[0.0, 0.5]$
Ground stiffness (kN/m) / ζ	rigid	$[1.3, 6.5]$ / $[0.3, 1.0]$
Object added mass (kg) / friction	$[0.0, 0.5]$ / $[0.6, 0.8]$	$[0.0, 0.5]$ / $[0.6, 0.8]$
Proprioception and depth camera [51], [52]		
IMU $\boldsymbol{\omega}_b$ (rad/s) / gravity noise	± 0.20 / ± 0.05	± 0.20 / ± 0.05
Joint $\mathbf{q}$ (rad) / $\dot{\mathbf{q}}$ (rad/s) noise	± 0.01 / ± 0.50	± 0.01 / ± 0.50
Camera mount offset	± 2.5 cm, $\pm 2.5^\circ$	± 2.5 cm, $\pm 2.5^\circ$
Stereo speckle $\sigma(d)$ / bias (m)	$0.001\, d^2$ / ± 0.01	$0.001\, d^2$ / ± 0.01
Pixel dropout / frame delay	3% / 1–3	3% / 1–3
Occluders / side masking [52]	0–2 / ≤ 12 cols	0–2 / ≤ 12 cols

frequency $\omega_n = 10$ Hz (20π rad/s) and damping ratio $\zeta = 2.0$:

$$k_{p,j} = I_j \omega_n^2, \quad k_{d,j} = 2\zeta I_j \omega_n, \quad \alpha_j = \frac{0.25\, \tau_{\max,j}}{k_{p,j}}, \qquad (5)$$

where $\tau_{\max,j}$ is the joint torque limit, and the policy commands target positions $\mathbf{q}_t^{\text{tar}} = \mathbf{q}^0 + \boldsymbol{\alpha} \odot \mathbf{a}_t$. To match our physical Unitree G1 (37.56 kg with the rear Mac mini enclosure and head camera mount), we replay open-loop trajectories on the robot and fit the effective per-joint stiffness, damping, rotor armature, torque limits, and transport delay to the measured response. All training phases use this identified model with a delayed PD actuator.

Physical, ground, and depth randomization. Randomization is kept narrow in Phases 1 and 2 to preserve teacher fidelity, then widened in Phase 3 (Table VIII). Phase 3 randomizes PD gains, rotor armatures, torque limits, link masses, and randomized command latency (15–40 ms) around the identified hardware model. We also randomize object mass and friction, as well as ground friction and contact compliance (stiffness 1.3–6.5 kN/m and damping ratio $\zeta \in [0.3, 1.0]$) so the policy transfers across both rigid floors and compliant carpets. Simulated depth frames in Phases 2 and 3 are corrupted with camera-mount jitter (± 2.5 cm, $\pm 2.5^\circ$), random rectangular occluders, distance-dependent stereo noise ($\sigma(d) = 0.001\, d^2$), 3% pixel dropout to the far plane, 1–3 frames of latency, and side masking [52] of up to 12 peripheral columns.

D. Orchestrator

Model. The orchestrator runs on the Mac mini using Qwen3.6-35B-A3B (4-bit, MLX). It shares one serving engine with the motion generator (Appendix E.1), so planning needs no network connection. It can also call frontier closed-source models through their API, which may give better planning performance.

Subtask descriptions. The orchestrator learns what the motion generator can do from a description of each subtask in its prompt: when the subtask applies and what its completion looks like. When a subtask is selected, the generator receives the subtask's text prompt. Table X lists the subtasks and their generator prompts; the full descriptions are in the plan-authoring prompt below.

Plan decomposition. When an instruction arrives, the orchestrator looks at the current egocentric frame and breaks the instruction into an ordered plan of up to eight subtasks from Table X, together with a short reply to the operator. The output follows a fixed JSON schema, so every step names a subtask the robot has. Each call sends a fixed system prompt, which the serving engine caches, and a user message with the operator's request and the newest egocentric frame. An example is shown below.

```
SYSTEM PROMPT:
You are a humanoid robot that performs whole-body motion
  subtasks. Understand the request, plan an ordered task
  list of subtasks, execute it, and reply to the human in
  one short sentence (no numbers or measurements). If the
  request needs no action, just reply. Never report
  results you did not observe.
SUBTASKS:
  go_to_chair() - walk to a chair across clear floor and
    stop about a metre in front of it. Use before sit.
      done when: the chair is close and in view.
  go_to_box() - walk to a box across clear floor and stop
    in front of it.
      done when: the box is close and in view.
  go_to_human() - walk to a person across clear floor and
    stop.
      done when: the person is close and in view.
  follow() - stay with a target that can move away, such
    as a person leading the way.
      done when: never on your own judgment; the operator
        ends it.
  sit() - sit down on a seat directly ahead with a clear
    path.
      done when: the robot is seated.
  crouch_under() - pass under an obstacle by bending the
    knees. The default for getting under a table.
      done when: the robot is out the far side and upright.
  crawl_through() - crawl under an obstacle too low to
    crouch under.
      done when: the obstacle is behind the robot.
  sidewalk() - side-step through a half-open door or a
    tight gap.
      done when: the robot is through the gap.
  push_box() - push a box directly in front of the robot
    toward the goal.
      done when: the box has visibly moved.
  stand() - return to the standing pose and wait.
      done when: the robot is upright and still.
  hold() - keep the current pose; nothing to act on yet.
      done when: what was awaited has arrived.
USER MESSAGE:
  Crouch under the table, then sit on the chair.
  ego frame: now
  <current egocentric image>
```

Transitions. While a step runs, the orchestrator makes step decisions back to back, at its maximum rate. At each decision it looks at the current frame and chooses one action: the step is done (`complete_step`), the whole task is done (`finish_mission`), retry the current step (`retry_step`), insert a subtask to clear a blocked step

(insert_step), rewrite the plan (set_plan), or ask the operator (ask_operator). An example is shown below.

```
SYSTEM PROMPT:
You are a humanoid robot that performs whole-body motion
  subtasks. Your responsibility is to supervise the plan
  while it runs: at each check, look at the current image
  and decide the next action by calling ONE tool:
  set_plan(steps) - write or rewrite the plan.
  complete_step(success) - the current step is finished;
    advance.
  retry_step() - the step stopped early and its goal is
    still ahead; run it again.
  insert_step(subtask) - the step is blocked; run another
    subtask first, then resume.
  ask_operator(question) - you are unsure or lost sight of
    the target.
  finish_mission() - the whole task is done.

GENERAL RULES:
  - Judge each step from the current image, with the
    current pelvis height as support.
  - Prefer retry_step while the goal is still ahead and
    reachable, insert_step when a different motion is
    needed to clear the way, and ask_operator rather than
    guessing when the scene is ambiguous.
  - A step toward a static object is done once the robot
    has arrived and stopped.
  - A step that follows a moving target is never done on
    your own judgment: keep following while the target is
    visible, ask the operator if you lose sight of it, and
    let only the operator end it.
  - Complete a step only when the robot has stopped and is
    back in a stable pose; while it is still moving or low,
    only observe, insert a step, or ask.

USER MESSAGE:
  Task: Crouch under the table, then sit on the chair.
  Plan:
    1. [completed] crouch_under - crouch under the table
    2. [in_progress] go_to_chair - walk to the chair
    3. [pending] sit - sit down on the chair
  NEXT step: 'sit' (acts on the chair). If the robot is
    already in a good position to start it (the chair is
    close and in view), call complete_step.
  This step goes to a STATIC object: once the robot has
    ARRIVED and stopped, the step is done - call
    complete_step to advance to the next step.
  Robot pose (raw): base_height=0.78m, yaw=-165deg
  ego frame: now
  <current egocentric image>
```

E. System

Software. The tracker runs as a C++ process on the Jetson; the generator and the orchestrator run as separate processes on the Mac mini. An optional web GUI shows the robot state, the current plan, and the orchestrator's decisions. The same processes run in MuJoCo and on the robot; only the tracker binary and the camera source change. To enforce strict real-time determinism, the tracker is implemented in C++ on a PREEMPT_RT kernel, pinned to isolated CPU cores under SCHED_FIFO priority with zero dynamic heap allocation, and coupled to the motor interface via lock-free triple buffers that prevent blocking across rate boundaries.

Processes talk through halo_comm, our lightweight middleware built on ZeroMQ and protobuf, with matching Python and C++ versions. It supports request/reply services and publish/subscribe topics. Subscribers keep only the newest message, so a slow process always sees the latest state rather than a backlog. A registry on the robot's Jetson lets processes on different machines find each other by channel name. Figure 7 shows the processes and channels.

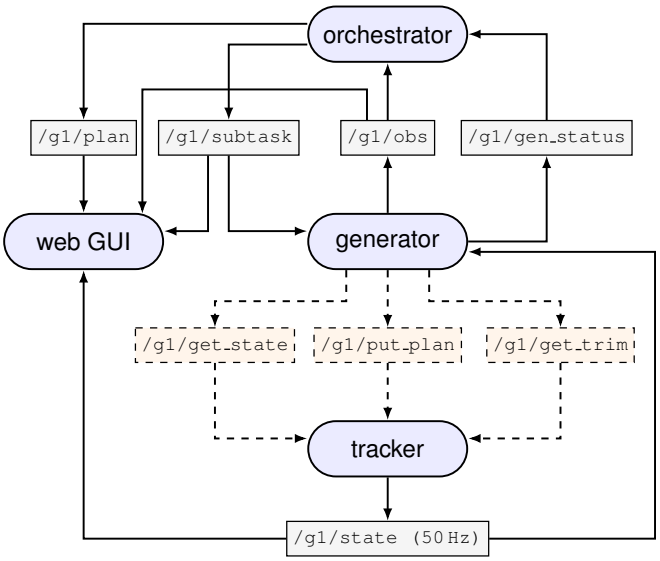


Fig. 7: **Communication graph.** The generator pulls the robot state (get_state) and the not-yet-executed reference (get_trim) from the tracker and sends each new motion plan (put_plan). The orchestrator sends the current subtask to the generator (subtask) and receives the egocentric frame (obs) and the generator's motion status (gen_status). The web GUI subscribes to state, obs, subtask, and plan.

Serving engine. The generator and the local orchestrator share the Mac's GPU through one scheduler that runs all GPU work on a single thread. Motion plans have priority (333 ms deadline) and can pause an orchestrator call between model layers, but the orchestrator keeps at least 60% of GPU time so it is never starved. The fixed part of the orchestrator prompt is cached, so each call only processes the new part. The generator's DiT and Qwen3.5-2B encoder run in MLX.

Latency. Table IX summarizes the measured latency of the onboard stack on the Mac mini M4 Pro.

TABLE IX: **Onboard latency on the Mac mini M4 Pro.** Motion generator latency is measured only for the model inference. Orchestrator latency is measured per transition decision, with the fixed prompt prefix cached.

Stage	Latency
Motion generator, alone	∼152 ms
Motion generator, co-served with orchestrator transition	∼155 ms
Orchestrator transition, alone	∼0.72 s
Orchestrator transition, co-served with generator	∼1.46 s

Hardware. Our hardware platform (Fig. 8) is a Unitree G1 humanoid modified for fully untethered operation with two physical additions: a rigid head mount carrying a forward-facing OAK-1W wide-angle camera, complementing the factory downward-facing RealSense D435i, and a rear protective enclosure housing a Mac mini M4 Pro (64 GB

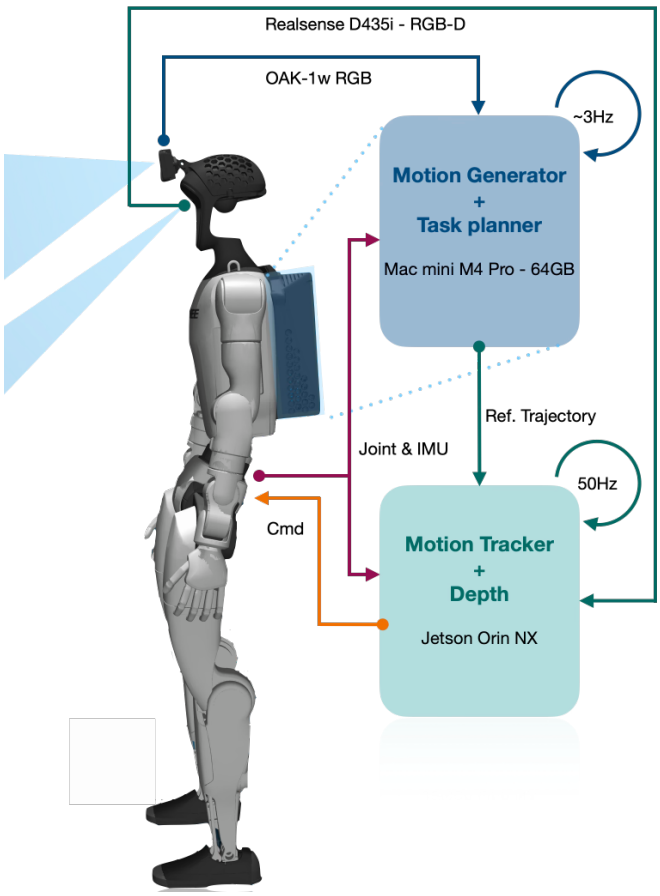


Fig. 8: **Onboard compute and sensing.** A Mac mini M4 Pro in the rear enclosure runs the orchestrator ($\sim 0.5\,\mathrm{Hz}$) and the motion generator ($\sim 3\,\mathrm{Hz}$). The robot's Jetson Orin NX runs the motion tracker in a $50\,\mathrm{Hz}$ real-time loop. The OAK-1W supplies the $120°$ RGB the generator reasons over; the RealSense D435i supplies the $55°$ depth the tracker reacts to. All modules run onboard, with external network calls used only when querying an optional cloud VLM for orchestration.

unified memory). Powered directly from the robot's internal power bus and bridged to the factory Jetson Orin NX via gigabit Ethernet, this dual-compute architecture isolates high-capacity planning from time-critical motor control in a two-rate asynchronous cascade over ZeroMQ. In the outer loop, the OAK-1W provides a $120°$ RGB stream (1280×80, $30\,\mathrm{Hz}$) to the Mac mini, where the orchestrator and motion generator synthesize kinematic reference trajectories onboard, with off-robot communication restricted to optional cloud VLM queries. In the inner loop on the Orin NX, the controller tracks this trajectory while ingesting near-field geometry from the factory $55°$ downward-facing Intel RealSense D435i (424×240, $30\,\mathrm{Hz}$). We bypass the D435i's noisy onboard depth pipeline altogether: we disable the active IR projector and route the raw stereo IR pair directly to a TensorRT-accelerated LiteAnyStereo [45] model co-resident on the Orin, recovering clean depth at $30\,\mathrm{Hz}$ without occlusion

TABLE X: **Subtasks available to the orchestrator.** *Generator prompt* is the text sent to the motion generator when the subtask is selected; the full subtask descriptions are in the orchestrator's system prompt (Appendix D). `hold` and `stand` do not run the generator model: the system sends a standing or pose-holding reference built from the tracker's current reference.

Subtask	Generator prompt
`go_to_chair`	*walk to the chair*
`go_to_box`	*walk to the box*
`go_to_human`	*walk to the person*
`follow`	*walk to the person*
`sit`	*sit down on the chair*
`crouch_under`	*crouch under the table*
`crawl_through`	*crawl under the desk*
`sidewalk`	*side-step through the narrow gap*
`push_box`	*push the box*
`stand`	—
`hold`	—

shadows or speckle noise and raising real sensor fidelity to match the simulator.

F. Experimental details

Metric for evaluation in simulation. We evaluate in MuJoCo with the full deployment stack. The generator sees the head-camera view rendered with the OAK-1W fisheye model. Simulation runs in real time, so planning latency is included.

Success criteria. A rollout fails if the robot falls at any point: pelvis tilt above $60°$ or pelvis height below $0.30\,\mathrm{m}$ for at least $1\,\mathrm{s}$. Otherwise it succeeds if its task's criteria hold.

- CROUCH and CRAWL: the robot passes under the table (not around it) without colliding with it, then stands up and stops beyond it.
- DOOR: the robot passes through the doorway into the room without colliding with or crossing the door leaf, then stands still inside, clear of the door's swing.
- SIT: the robot ends seated within $0.9\,\mathrm{m}$ of the chair (pelvis at $0.35\,\mathrm{m}$ to $0.68\,\mathrm{m}$, torso within $40°$ of upright).
- PUSH: the box moves at least $0.3\,\mathrm{m}$ during the push.
- GO-TO: the robot stops within $1.0\,\mathrm{m}$ of the named object.
- Multi-task sequences: every subtask succeeds, in order.

Real-world rollouts. Figure 9 shows one real-world rollout per skill, and Figure 10 shows the same skills in other rooms, with other people and with other objects. All runs use the hardware of Appendix E.2.

G. Discussion

Frozen vs. fine-tuned VLM. The motion generator in Appendix B keeps the VLM encoder frozen. We also tried fine-tuning it, updating both Qwen's vision encoder and its language model (learning rate 10^{-5} for the VLM, 10^{-3}

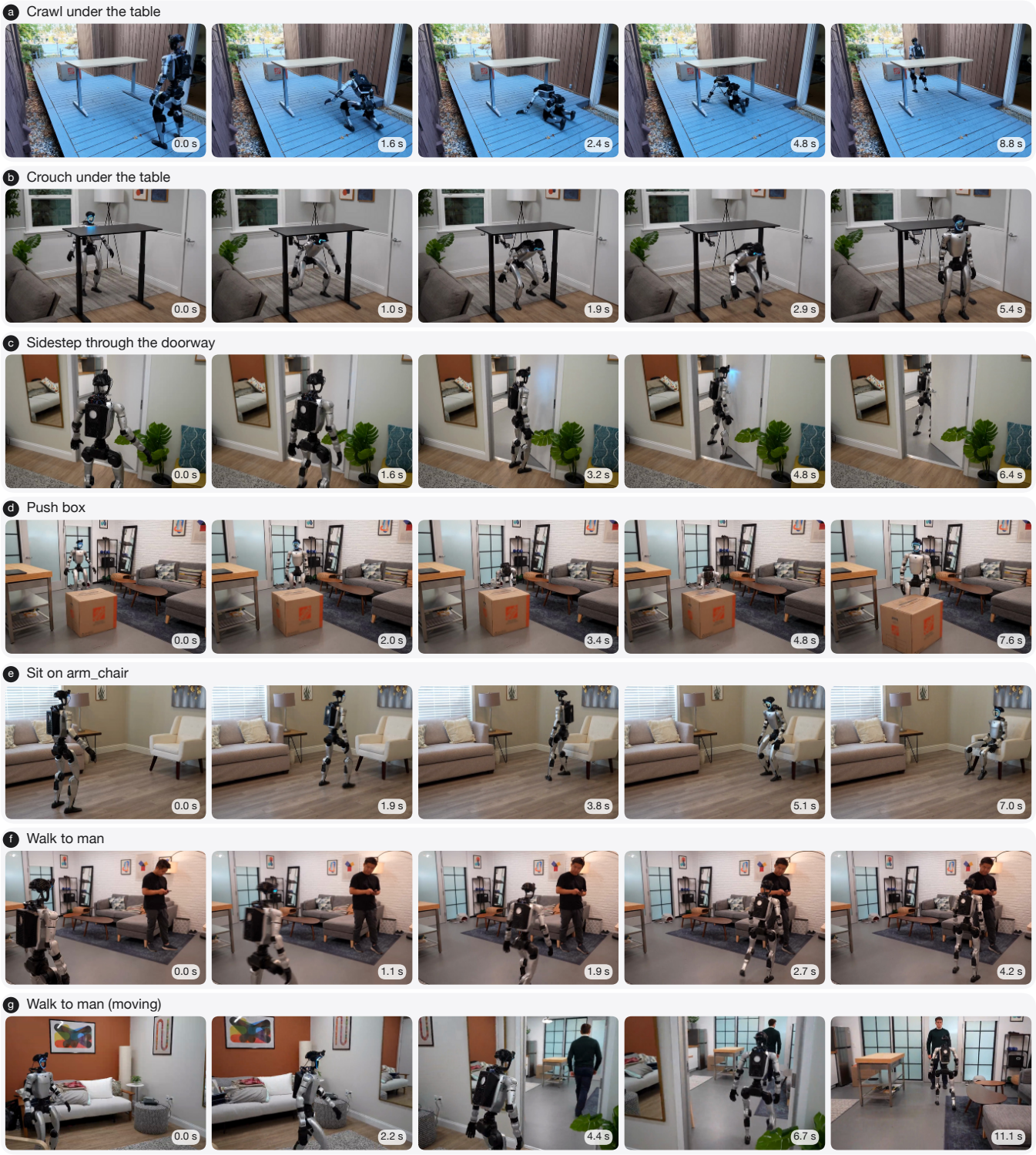


Fig. 9: **Real-world rollouts, one per skill.** Each row is one single-task run on the physical robot, filmed by an external camera. The row header is the subtask prompt given to the motion generator. The five frames are spaced through the run, and the time in each frame is measured from the first frame of its row.

for the decoder). We did not study this systematically; the observations below come from training these variants and deploying them on the robot.

Responsiveness. With a fine-tuned encoder, the robot reacts faster and moves more smoothly, most visibly when following a person. Our explanation is that a frozen VLM knows *what* is in the image but not precisely *where*. For example, a person at a bearing of 39° and one at 40° produce almost the same features, so the generator cannot respond to small movements. Fine-tuning teaches the encoder to keep

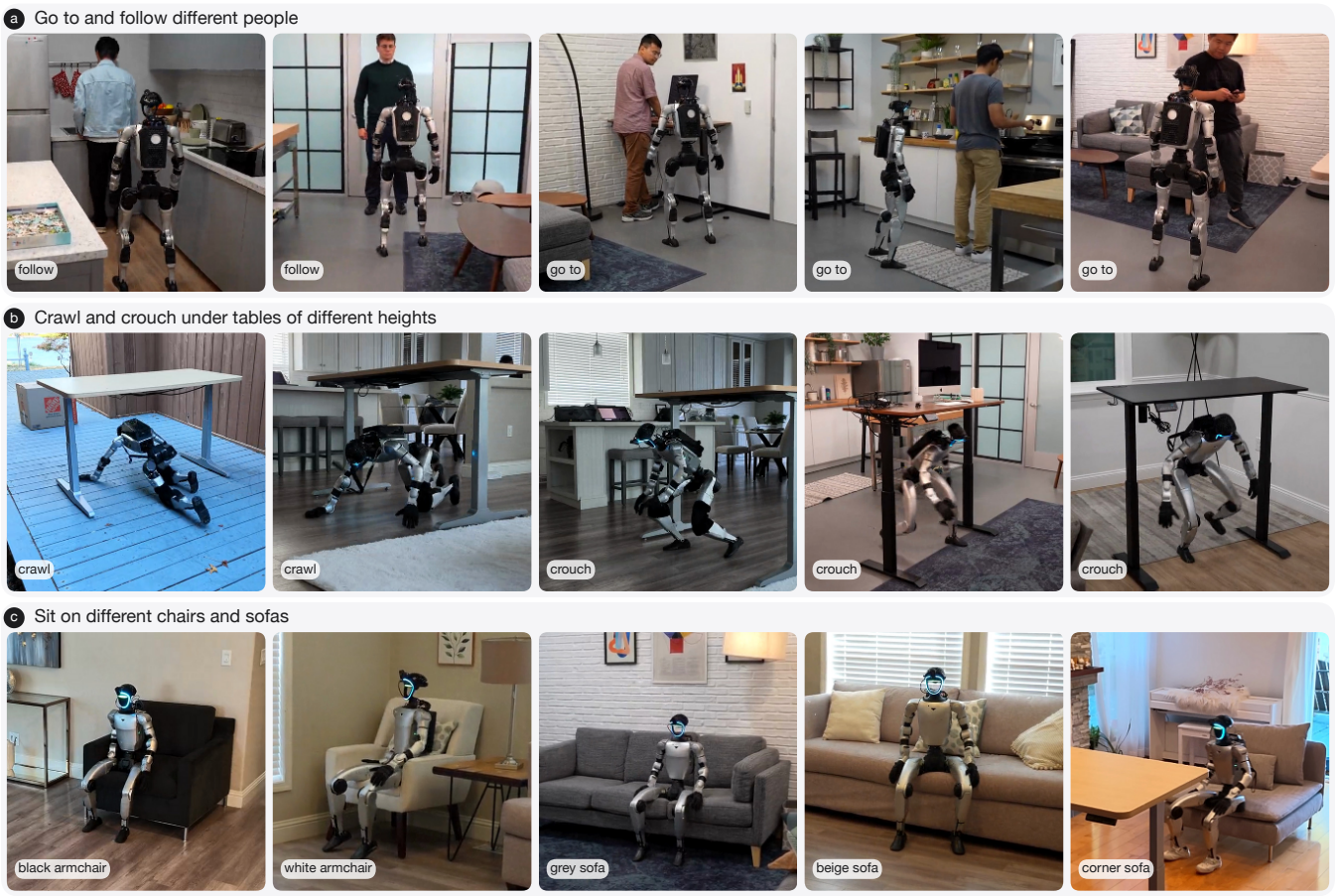


Fig. 10: **Real-world generalization across people, tables, and seats.** Each tile is one frame from a separate real-world run. (a) GO-TO toward different people in different rooms. (b) CRAWL under two tables (outdoors and indoors) and CROUCH under three desks of different heights. (c) SIT on a black armchair, a white armchair, two sofas and a corner sofa.

this fine spatial detail.

Generalization. Our training data for following a person contained only one human model. With a frozen encoder, the robot still followed many different people in the real world, because the VLM already understands "person" in general. After fine-tuning, the robot only recognized people who looked like that one model. To fix this, we re-rendered these episodes with many different human models, created with the 3D asset generator Hunyuan3D [56]. With varied people in training, the fine-tuned model generalizes again.

Instruction following. In our data, the image often already reveals the task: if a chair is in view, the command is usually "*sit on the chair*". A fine-tuned model learns this shortcut, relying on the image and ignoring the command. To fix this, we replace the image with a black frame on 30% of training samples. On those samples the model can only succeed by reading the command, so it learns to follow the language command again.

Motion tracker design and deployment. Deploying the motion tracker across different homes and outdoor scenes exposed several physical gaps that a standard simulator with datasheet parameters and rigid floors does not capture. The observations below come from comparing these design choices on the robot.

System identification. Off-the-shelf simulation models of the G1 omit the internal battery, underestimate hip rotor inertia, and assume ideal, delay-free PD drives. On hardware, the physical motors deliver only 78–90% of the commanded stiffness with a 25–30 ms end-to-end loop delay, and the ankle and waist parallel linkages exhibit higher effective inertia and friction than the serial joints. Fitting these parameters from open-loop hardware trajectories cuts joint tracking error on held-out robot motions by 62% (26.9% to 10.1%) and reduces short-horizon prediction error during walking by about a third. More importantly, centering training on the identified robot lets Phase 3 domain randomization use narrow, realistic ranges around the true dynamics instead of wide bands that make the policy overly cautious.

Floor and carpet randomization. Our real-world evaluations span wooden and glossy laminate floors, tile, indoor and outdoor cement, doorway thresholds, and carpets of varying thickness, often within a single multi-task rollout. A policy trained only on rigid ground transfers well to hard surfaces (wood, tile, and cement), but struggles on plush carpet: the robot's weight sinks the feet by 1–2 cm into the pile, which a rigid-floor policy misinterprets as a tilt disturbance and tries to correct by continuously shuffling its feet or catching a toe when turning in place. Conversely, without low-friction

randomization, the feet slip during deceleration and turns on dusty wood or smooth laminate. Randomizing both surface friction and ground contact compliance (stiffness and damping), together with the Phase 3 stance foot-slip penalty, teaches the robot to stand quietly on thick carpets while keeping firm footing on slippery hard floors.

Action smoothness and latency. Across trained checkpoints, raw simulation reward correlates weakly with real-world stability, whereas the action-rate smoothness penalty strongly predicts clean hardware transfer. Without sufficient action-rate regularization and randomized command latency (15–40 ms), high-frequency policy actions interact with hardware transport delay and joint stiction, causing audible vibration at the ankles and wrists.